JAVA SDE-2 INTERVIEW PREPARATION SERIES

JAVA ENGINEERING - BOOK 07 OF 09 - STUDY STEP JAVA-07

Java Concurrency and the Memory Model

Threads, Happens-Before, Locks, Atomics, Executors, Virtual Threads, and Failure Modes

BY

Vinay Reddy Kalluri

Reason from happens-before and ownership through locks, atomics, task execution, concurrent collections, virtual threads, testing, and cancellation.

JMM | THREADS | LOCKS | ATOMICS | EXECUTORS | VIRTUAL THREADS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `JAVA 06 - JVM and Execution`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This book develops concurrency correctness from first principles and then connects it to production scheduling, cancellation, backpressure, and testing.

Readiness map

Decision	Evidence
START HERE	Multiple threads access mutable state; Visibility and ordering are being confused with atomicity; Tasks outlive requests or need cancellation; Thread count is mistaken for capacity.
PREPARE FIRST	JAVA-06 runtime model; Collections and exception fundamentals.
MOVE ON WHEN	Use happens-before to explain visibility; Choose locks, atomics, or immutability; Design executor and cancellation policies; Test and diagnose concurrency failure modes.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Java Segment Position

JAVA 06 - JVM and Execution	**YOU ARE HERE** **JAVA 07 - Concurrency and Memory Model**	JAVA 08 - Performance and Diagnostics

A note from Vinay

Concurrency becomes manageable when you state the invariant and the happens-before edge explicitly. If neither is clear, adding another lock or concurrent collection is guesswork.

Contents

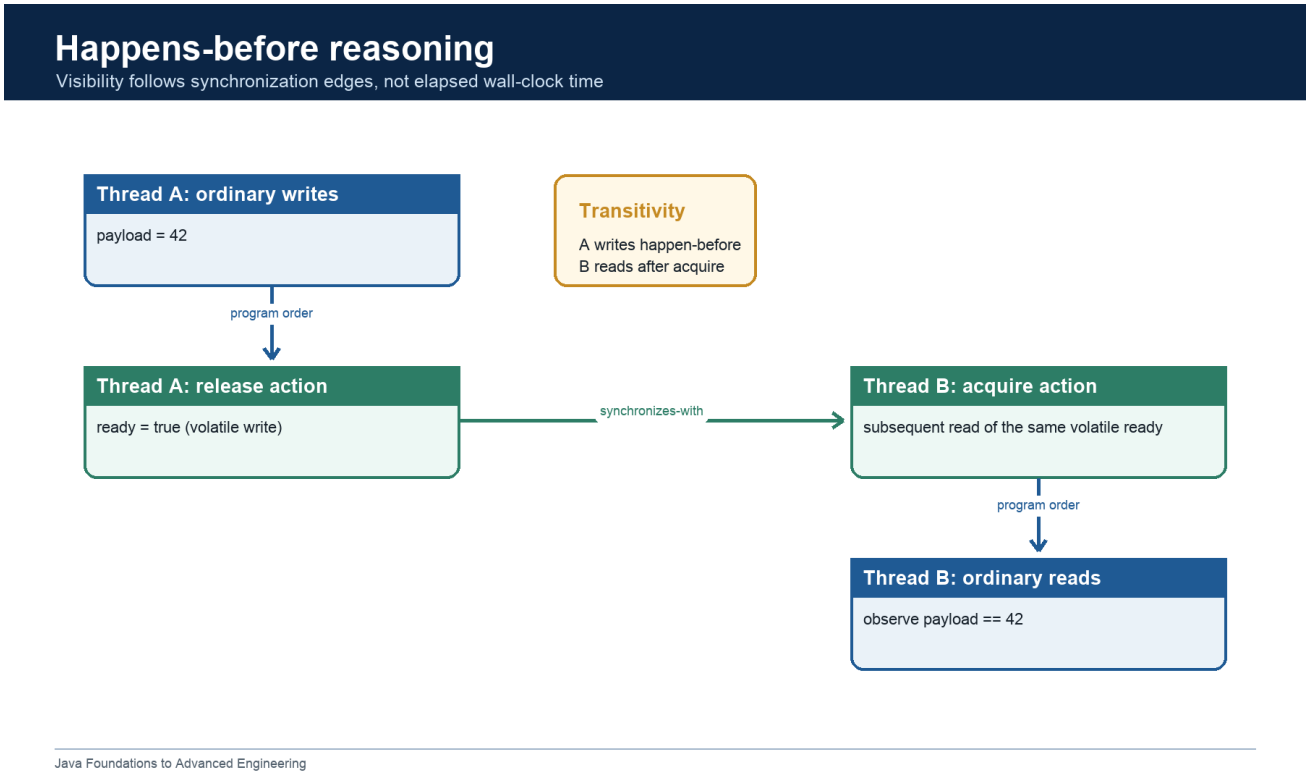
This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - The Java Memory Model from First Principles



A happens-before chain through the same volatile variable

Learning objectives

- Separate atomicity, visibility, and ordering in concurrent Java.
- Use happens-before to prove which writes a read is guaranteed to observe.
- Explain monitor, volatile, thread start/join, task synchronization, default initialization, and final-field rules.
- Understand data races, safe publication, immutable objects, and broken double-checked locking.
- Avoid reasoning only from source interleavings or a particular CPU cache story.

WHY IT WORKS | Why this matters at SDE-2

Concurrency bugs often pass tests and fail under production load, another architecture, or a new JIT. Locks are not merely mutual exclusion. They, volatile operations, and lifecycle APIs establish ordering that makes writes visible. Without such relationships, "thread A ran first" is not a proof.

An SDE-2 should be able to review a publication pattern, explain why `volatile` does not make `count++` atomic, select a synchronization mechanism, and state the proof in happens-before terms. This skill underlies concurrent collections, executors, atomics, virtual threads, and reliable service shutdown.

WHY IT WORKS | First-principles model

Each thread executes actions governed by its program order, but compilers and processors may transform implementation order when a single thread cannot tell the difference. Multiple threads can observe effects only within constraints imposed by the Java Memory Model (JMM).

The JMM is not a literal diagram of each core copying values to "main memory." It is a formal contract defining legal observations. The central proof relation is happens-before (HB): if action A happens-before action B, then A's effects are ordered before B, and conflicting memory observations must respect that order.

TEXT

```
writer thread                          reader thread
data = 42;                             if (ready) {
ready = true;  // volatile W ----HB----> volatile R of true
                                       use(data); // guaranteed to see 42
                                   }
```

The volatile write of `ready` synchronizes-with a subsequent volatile read that observes the relevant write in the volatile order. Program-order edges place `data=42` before that write and the `data` read after the volatile read. HB transitivity orders the data write before the data read.

Core terminology

- **Memory action:** Read/write of a variable, volatile action, lock/unlock, thread lifecycle action, and other JMM-relevant operation.
- **Program order:** Per-thread order consistent with intra-thread semantics.
- **Synchronization action:** Operation participating in the global synchronization order, such as volatile access or monitor lock/unlock.
- **Synchronizes-with:** Specified cross-thread edge created by matching synchronization actions.
- **Happens-before:** Transitive ordering built from program order and synchronizes-with plus other specified rules.
- **Data race:** Two conflicting accesses to the same variable, at least one a write, not ordered by happens-before.
- **Sequential consistency (SC):** Behavior explainable by one total interleaving preserving each thread's program order.
- **Safe publication:** Making an object reachable by other threads through an ordering mechanism that exposes required initialized state.
- **Visibility:** Guarantee about which writes a read can observe.
- **Atomicity:** Indivisibility of an operation relative to other threads.
- **Ordering:** Constraints preventing observable reordering across actions.

Detailed mechanics

Three properties must be separated:

- **Atomicity:** A read or write may be indivisible, yet a read-modify-write expression is multiple actions. `count++` reads, adds, and writes; two threads can lose an update.
- **Visibility:** One thread may not be guaranteed to observe another thread's ordinary write without HB, even if wall-clock time passed.
- **Ordering:** Operations can be implemented in a different order when allowed, and another thread with a data race can observe results not predicted by a naive source interleaving.

Reads and writes of references and most primitive values are atomic under JMM rules. The specification still permits a non-volatile `long` or `double` access to be implemented as two 32-bit accesses, so a racing read could theoretically tear; volatile `long` and `double` accesses are guaranteed atomic. Mainstream 64-bit HotSpot platforms commonly perform aligned 64-bit accesses atomically, but correctness must not rely on that implementation fact. Atomic individual access still does not make compound invariants atomic. Array elements and fields are distinct variables, and implementations must prevent a write to one from tearing into an adjacent variable.

Happens-before includes these crucial rules:

1 Every action in a thread happens-before each later action in that thread's program order.

2 An unlock of a monitor happens-before every subsequent lock of that same monitor.

3 A write to a volatile variable happens-before every subsequent read of that variable in the synchronization order.

4 A call to `Thread.start()` happens-before actions in the started thread.

5 All actions in a thread happen-before another thread successfully returns from `join()` on it.

6 Default initialization of an object happens-before any other actions, providing zero/default state safety.

7 HB is transitive.

Library specifications add edges. For example, executor submission, `Future.get`, concurrent collections, latches, and queues state memory-consistency effects. Use those documented contracts; do not assume that "it uses threads" provides publication.

Monitors provide both mutual exclusion and ordering. Exiting a `synchronized(lock)` block unlocks; later entering a block on the same monitor locks. Synchronizing on different objects creates no matching edge. Reentrancy lets a thread reacquire a monitor it owns but does not weaken cross-thread rules.

Volatile provides ordering/visibility for a variable without making arbitrary multi-step invariants mutually exclusive. A volatile read sees some write consistent with volatile synchronization order; if it sees a publication flag's write, earlier writer effects are visible after that read. A volatile write has release-like effects and a volatile read acquire-like effects as an intuition, while the formal Java rule remains synchronizes-with/HB.

Volatile is suitable for independent state such as a shutdown flag or immutable snapshot reference. It is insufficient for `if (stock > 0) stock--`, `counter++`, or updates across multiple fields that must be observed consistently. Use a lock, an atomic read-modify-write primitive, or an immutable state transition through CAS.

Data-race-free programs whose synchronization is correctly used receive a powerful guarantee often summarized as DRF-SC: executions appear sequentially consistent. This is why synchronization should define the program, not merely add barriers after a bug appears. A data race does not mean "the latest value eventually appears"; allowed observations can be unintuitive while still constrained by causality and safety rules.

Final fields have special initialization safety. If an object's constructor completes and the reference does not escape during construction, another thread that obtains the reference, even through some racy forms, receives specified guarantees for correctly constructed final fields and reachable state covered by the final-field semantics. This is not a license for racy publication: non-final fields, later mutations, compound object graphs, and the reference itself still demand robust publication. Use final fields plus safe publication.

Safe publication idioms include:

- constructing before storing into a `volatile` reference;
- storing while holding a monitor that readers also acquire;
- static initialization/class initialization;
- placing into a concurrent collection or synchronization-aware queue;
- passing state before `Thread.start()`;

- completing a `Future`, promise-like API, or latch under its documented memory effects.

Thread confinement avoids sharing entirely. Immutability reduces proof surface because state cannot change after construction. Neither helps if mutable components leak or an object's constructor publishes `this` prematurely.

Double-checked locking is correct only with a volatile publication field in its standard modern form:

JAVA

```java
private static volatile Service instance;

static Service instance() {
    Service local = instance;
    if (local == null) {
        synchronized (Service.class) {
            local = instance;
            if (local == null) {
                local = new Service();
                instance = local;
            }
        }
    }
    return local;
}
```

Without volatile, publication can race with construction effects, and the unsynchronized read has no required HB connection. Simpler static-holder initialization is often preferable.

> **Specification boundary:** The JMM is part of Java language semantics. It defines allowed observations through actions and ordering relations, not cache-flush instructions, CPU fences, or HotSpot compiler phases. A correct proof should remain valid across processors and conforming JVMs.

TRACE IT | Worked Java example

JAVA

```java
public final class Publication {
    private int payload;
    private volatile boolean ready;

    void publish(int value) {
        payload = value;
        ready = true;
    }

    int await() {
        while (!ready) {
            Thread.onSpinWait();
        }
        return payload;
    }

    public static void main(String[] args) throws InterruptedException {
        Publication publication = new Publication();
        Thread reader = new Thread(() ->
                System.out.println(publication.await()));
        reader.start();
        publication.publish(42);
        reader.join();
    }
}
```

If `await` terminates after reading `ready == true`, it must return 42. The volatile edge publishes the earlier ordinary write. `Thread.onSpinWait()` is only a hint and supplies no visibility guarantee; `volatile` supplies the ordering.

This is an educational pattern, not a recommended blocking design. Busy waiting consumes a carrier/CPU resource. A latch, queue, future, or higher-level synchronizer usually expresses waiting more efficiently.

TRACE IT | Execution or memory walkthrough

Name the actions:

TEXT

```
Main/writer                          Reader
S: reader.start()
P: payload = 42
W: volatile ready = true
                                     R: volatile read ready == true
                                     Q: read payload
                                     T: reader terminates
J: reader.join() returns
```

Proof:

1 S happens-before actions in the started reader, though this alone does not publish the later write P because P occurs after start.

2 Writer program order gives `P HB W`.

3 The volatile rule gives `W HB R` for the subsequent read of the published true write.

4 Reader program order gives `R HB Q`.

5 Transitivity gives `P HB Q`, so Q must observe payload=42 or a later HB-consistent write, and none exists here.

6 All reader actions happen-before `J`, so after `join`, the main thread can safely observe reader effects as defined.

If `ready` is ordinary, P and W are not connected to reader R/Q by synchronization. Source order and physical elapsed time are insufficient. The loop could fail to terminate, or the reader could observe an allowed stale/independently ordered state.

If two writers call `publish` concurrently, volatile does not make the pair `(payload, ready)` an atomic transaction. The design assumes a single publication event. For repeated messages, use a queue, lock, sequence protocol, or immutable snapshot held in one volatile reference.

Complexity and performance

Each volatile read/write is O(1) algorithmically, but its constant cost includes compiler and hardware ordering constraints and cache-coherence traffic. An uncontended monitor can also be fast, while contention introduces queueing, park/unpark, scheduler delay, and convoy effects. Big-O alone is not useful for synchronization choices.

The spin loop performs O(k) reads until publication and consumes CPU while waiting. With no publication, time is unbounded. Blocking synchronization trades wake-up latency for released CPU. Hybrid spin-then-park strategies belong in well-tested concurrency libraries, not ad hoc business code.

False sharing is an implementation/hardware performance issue where independent frequently written variables share cache-coherence granularity. It does not change JMM correctness. Padding or special annotations are JVM-specific optimization techniques requiring measurement.

> **HotSpot note:** HotSpot maps JMM requirements to compiler barriers, machine instructions, cache-coherence protocols, monitor implementations, and runtime stubs appropriate to the target CPU. x86-64 and AArch64 can require different instruction sequences. Those mappings are not portable source-level explanations.

WATCH OUT | Edge cases and common mistakes

- Saying volatile makes a variable "thread-safe" without defining the invariant.
- Using `volatile int count` and expecting `count++` to be atomic.
- Synchronizing writers but allowing readers to access the same fields without the same lock or another HB edge.
- Locking on different objects, mutable lock references, boxed values, or publicly accessible strings.
- Assuming `sleep`, logging, a debugger, or elapsed time flushes memory.
- Treating `ConcurrentHashMap` safety as automatically making mutable values safely updated under arbitrary operations.
- Starting a thread from a constructor and leaking partially constructed `this`.
- Assuming final reference means deeply immutable graph.
- Using double-checked locking without volatile.
- Believing data races merely return "old or new" values in every compound scenario.
- Reasoning from one observed run as proof.

Publication and mutation are separate. Putting a mutable `ArrayList` into a concurrent map can safely publish the list reference according to the map contract, but unsynchronized mutations of that list can still race. Similarly, `Collections.synchronizedList` requires its documented locking discipline during compound iteration.

Deadlock freedom and memory visibility are separate properties. A correct HB proof can still deadlock; a lock-free algorithm can still be logically wrong. Progress properties such as obstruction freedom, lock freedom, wait freedom, fairness, and starvation need separate analysis.

Production engineering notes

Prefer ownership and higher-level primitives:

- immutable request/config snapshots published atomically;
- executor task boundaries and futures;
- blocking queues for producer-consumer transfer;
- concurrent maps with atomic methods such as `compute` for key-scoped transitions;
- atomics for small independent state machines;
- locks for multi-field invariants;
- structured lifecycle with interruption and join/close semantics.

Document invariants next to guarded fields, for example "guarded by `lock`" or "immutable snapshot published through volatile `current`." Code review should identify every conflicting access and the HB path ordering it, not only verify that some `synchronized` appears.

Race testing is probabilistic. Stress tools, randomized scheduling, high iteration counts, and multiple architectures can expose bugs but cannot prove absence. Static analyzers and disciplined design help. Thread dumps reveal blocking/deadlock states but not all memory-order races. JFR and profiles can reveal lock contention; they do not replace a correctness proof.

When optimizing synchronization, first preserve a written invariant and establish a benchmark representing contention. Replacing a lock with multiple atomics can weaken snapshot consistency. A single volatile immutable state object is often both clearer and faster than coordinating several volatile fields.

TRY IT | Interview questions and model answers

What does happens-before mean?

It is the JMM ordering relation used to guarantee visibility and constrain observations. It is built from per-thread program order, synchronizes-with edges such as monitor unlock-to-lock and volatile write-to-read, thread lifecycle rules, and transitivity. It is not simply wall-clock ordering.

What does volatile guarantee?

Individual volatile access is atomic and participates in a total synchronization order. A volatile write happens-before subsequent reads of that variable, publishing earlier writer actions to code after the read. Volatile does not make compound operations or multi-field invariants atomic.

How does `synchronized` affect memory?

It provides mutual exclusion for the same monitor. Unlocking that monitor happens-before a subsequent lock, so writes before unlock are visible to actions after the matching lock. Different monitors do not create that edge.

Why is `count++` unsafe even if reads/writes of `int` are atomic?

Increment is read, compute, write. Two threads can read the same old value and both write the same incremented value. Use `AtomicInteger.incrementAndGet`, a lock, or an appropriate aggregate concurrency design.

How do you safely publish an object?

Construct it without leaking `this`, preferably make state final/immutable, then publish through a specified edge: volatile write/read, matching monitor unlock/lock, class initialization, thread start, concurrent collection, queue, future, or another synchronizer's documented memory effect.

Does final make an object thread-safe?

No. Final prevents reassignment of that field after construction and supplies special initialization safety for correctly constructed objects. The referenced object may still be mutable, and later mutations require synchronization.

Why can a racy loop fail to see a flag update?

Without synchronization, there is no HB edge requiring the read to observe the other thread's write. Compiler optimizations and hardware execution may reuse or reorder values within JMM permissions. Volatile or a synchronizer establishes the required relation.

TRY IT | Exercises

1. Draw the HB graph for the worked example and remove each edge in turn.
2. Implement a lost-update counter using plain `int`, then correct it with a lock and an atomic.
3. Review a mutable object placed in `ConcurrentHashMap`; identify which operations remain unsafe.
4. Implement lazy initialization using the static-holder idiom and compare its proof with volatile double-checked locking.
5. Design an immutable configuration snapshot with one volatile reference.
6. Explain the memory effects of submitting a task and obtaining its `Future` result from library documentation.
7. Find a `this`-escape pattern in a constructor and redesign it with a factory.
8. State correctness, liveness, and performance properties separately for a proposed concurrent queue use.

RECAP | Chapter summary

The JMM defines legal cross-thread observations through actions and ordering relations, not a literal cache architecture. Atomicity, visibility, and ordering are distinct. Happens-before proofs connect program order with volatile, monitors, thread lifecycle, class initialization, and synchronization-library contracts. Data-race-free designs gain sequentially consistent reasoning. Safe construction, safe publication, immutability, ownership, and high-level synchronizers are the practical path to reliable concurrency.

TRY IT | Revision checklist

- [ ] I can distinguish atomicity, visibility, and ordering.
- [ ] I can define data race, synchronizes-with, and happens-before.
- [ ] I can list monitor, volatile, start, join, and initialization edges.
- [ ] I can prove the volatile publication example by transitivity.
- [ ] I know why `volatile count++` is unsafe.
- [ ] I can safely publish immutable and mutable state.
- [ ] I understand final-field guarantees and `this` escape limitations.
- [ ] I can explain double-checked locking and prefer simpler alternatives.
- [ ] I do not substitute CPU-cache folklore or tests for a JMM proof.

SDE-2 CORE CHAPTER

Chapter 2 - Threads, Lifecycle, Interruption, and Cancellation

Learning objectives

- Explain Java thread states as observable lifecycle categories rather than scheduler commands.
- Distinguish `start`, `run`, sleep, waiting, blocking, joining, daemon status, and termination.
- Use interruption as a cooperative cancellation protocol.
- Propagate or restore `InterruptedException` correctly.
- Design thread-owning components with explicit startup, bounded shutdown, and failure reporting.

WHY IT WORKS | Why this matters at SDE-2

Services fail during shutdown as often as during steady state. A worker that swallows interruption can delay deployment, lose a partition lease, or leave a process to be killed after its grace period. A thread created without ownership can outlive the component that created it. A daemon thread can disappear before flushing critical work. An uncaught exception can silently remove capacity if no one observes it.

At SDE-2, "interrupt stops a thread" is not acceptable. Interruption requests cooperation. The target decides when and how to stop, blocking methods define how they react, and ownership code must wait for termination or report that it could not. These rules apply later to executors, futures, virtual threads, and structured concurrency designs.

WHY IT WORKS | First-principles model

A thread is an independently scheduled sequence of Java actions sharing process resources with other threads. Creating a `Thread` object does not begin concurrent execution. Calling `start()` asks the runtime to schedule a new execution that invokes `run()` once.

TEXT

```
NEW --start()--> RUNNABLE --------------------------> TERMINATED
                  |   ^                                  ^
                  |   | monitor acquired / event         |
                  v   |                                  | run returns
               BLOCKED                                  | or throws
                  |                                     |
                  +--> WAITING / TIMED_WAITING ----------+
                       notify, unpark, timeout, interrupt
```

`RUNNABLE` includes threads actually executing and threads ready but waiting for CPU or certain native operations. Java's state enum is a diagnostic snapshot. It is not a complete OS scheduler model, and code should not coordinate correctness by polling another thread's state.

Cancellation is a protocol:

```text
owner requests cancellation
  -> publish cancellation state and/or interrupt
  -> worker notices at a cancellation point
  -> worker preserves invariants and releases resources
  -> worker terminates
  -> owner joins or otherwise observes completion
```

Core terminology

- **Platform thread:** A traditional Java thread commonly mapped closely to an operating-system thread.
- **Virtual thread:** A lightweight Java thread scheduled by the runtime; covered deeply in Master Chapter 37.
- **Interrupt status:** Per-thread boolean cancellation signal manipulated by `interrupt`, `isInterrupted`, and `interrupted`.
- **Cancellation point:** A place where code checks status or calls an interruptible blocking API.
- **Daemon thread:** Thread that does not by itself keep the JVM alive.
- **Join:** Waiting for another thread to terminate.
- **Uncaught exception handler:** Last-resort observer invoked when a thread exits because `run` threw an uncaught exception.
- **Cooperative cancellation:** Target code participates in stopping at safe boundaries.
- **Thread confinement:** Mutable state accessed by only one thread.

Detailed mechanics

`Thread.start()` has two important properties. It creates a new concurrent execution of `run`, and actions before `start` happen-before actions in the started thread. Calling `run()` directly is an ordinary synchronous method call on the current thread. A `Thread` instance can be started at most once; a second `start()` throws `IllegalThreadStateException` even after termination.

The six `Thread.State` values are:

- `NEW`: constructed but not started.
- `RUNNABLE`: executing or eligible to execute in the JVM/OS view.
- `BLOCKED`: waiting to enter a `synchronized` monitor.
- `WAITING`: waiting indefinitely through operations such as untimed `join`, `wait`, or `park`.
- `TIMED_WAITING`: waiting with a deadline, including `sleep`, timed `join`, timed `wait`, or timed park.
- `TERMINATED`: `run` completed normally or abruptly.

State can change immediately after observation. It is useful for thread dumps, not for writing `if (thread.getState() == WAITING) ...` protocols.

Calling `interrupt()` normally sets the target's interrupt status. If the target is in an interruptible blocking method such as `sleep`, `wait`, `join`, or many `java.util.concurrent` waits, that method usually throws `InterruptedException` and clears the status. The exception is not an error to log and ignore. It is a request arriving through the method's control flow.

There are two standard handling choices:

1. **Propagate:** A method that can abandon its work declares `throws InterruptedException`, performs required local cleanup, and lets its caller choose policy.
2. **Restore and stop/return:** Code at a boundary that cannot throw restores status with `Thread.currentThread().interrupt()` and exits or reports cancellation.

Swallowing the exception loses the request. Restoring status and blindly continuing the same interruptible loop can cause immediate repeated failures, so restoration must accompany a deliberate boundary decision.

`Thread.interrupted()` returns and clears the current thread's status. `currentThread().isInterrupted()` observes without clearing. This naming difference causes bugs. A CPU-bound loop should check status at a reasonable frequency:

JAVA

```java
while (!Thread.currentThread().isInterrupted()) {
    computeOneChunk();
}
```

Not every block responds to interruption. Monitor entry for `synchronized` is not interruptible. Some legacy or native I/O may require closing a resource or using an API-specific cancellation mechanism. `interrupt()` cannot make arbitrary code safe to stop.

`join()` waits for termination and establishes a memory edge: all actions in the terminated thread happen-before another thread successfully returns from `join`. A timeout only bounds the wait; it does not terminate the target. Code must check `isAlive()` or use a higher-level completion result after a timed join.

Daemon status must be set before start. The JVM can exit when only daemon threads remain; it does not promise daemon `finally` blocks complete. Critical persistence, commits, lease release, and telemetry flushes need explicit lifecycle coordination rather than daemon reliance.

An uncaught exception terminates that thread. The JVM consults the thread's handler, thread group behavior, or default handler. Logging is useful, but a robust component also converts worker loss into health state or supervisor action. Restarting blindly can create a crash loop or duplicate unsafe work.

> **Specification boundary:** The JLS and Java APIs define thread actions, lifecycle methods, interruption status, and happens-before rules. They do not promise fair scheduling, immediate execution after `start`, immediate response to interruption, an OS thread mapping, or a maximum shutdown time.

TRACE IT | Worked Java example

JAVA (continued 1/2)

```java
import java.time.Duration;
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.TimeUnit;

public final class OwnedWorker implements AutoCloseable {
    private final BlockingQueue<String> jobs = new LinkedBlockingQueue<>();
    private final Thread thread = new Thread(this::runLoop, "owned-worker");
    private volatile boolean stopping;
    private boolean started;

    public synchronized void start() {
        if (started) throw new IllegalStateException("already started");
        if (stopping) throw new IllegalStateException("already stopped");
        started = true;
        thread.start();
    }

    public synchronized void submit(String job) {
        if (stopping) throw new IllegalStateException("stopping");
        jobs.add(job);
    }

    private void runLoop() {
        try {
            while (!stopping) {
                String job = jobs.poll(1, TimeUnit.SECONDS);
                if (job != null) process(job);
            }
        } catch (InterruptedException interrupted) {
            Thread.currentThread().interrupt();
```

JAVA (continued 2/2)

```
        } finally {
            releaseWorkerResources();
        }
    }

    private void process(String job) {
        System.out.println(job);
    }

    private void releaseWorkerResources() {}

    public void stop(Duration timeout) throws InterruptedException {
        if (timeout.isNegative() || timeout.isZero()) {
            throw new IllegalArgumentException("positive timeout required");
        }
        synchronized (this) {
            stopping = true;
        }
        thread.interrupt();
        thread.join(Math.max(1, timeout.toMillis()));
        if (thread.isAlive()) {
            throw new IllegalStateException("worker did not stop in " + timeout);
        }
    }

    @Override
    public void close() throws InterruptedException {
        stop(Duration.ofSeconds(5));
    }
}
```

The object has explicit ownership. Construction does not leak `this` by starting a thread. `start()` is one-shot. Shutdown publishes `stopping`, interrupts the queue wait, joins with a bound, and reports failure instead of pretending the worker died.

TRACE IT | Execution or memory walkthrough

1. The owner constructs `OwnedWorker`. The internal thread is `NEW`; no concurrent access begins.
2. Synchronized `start` marks the component started and calls `Thread.start`. Prior construction actions are visible to the worker through the start happens-before edge.
3. `runLoop` polls the thread-safe queue. With no work, the worker is generally `TIMED_WAITING` in the queue implementation.
4. `submit("A")` publishes the element through `BlockingQueue`'s memory-consistency contract. The worker obtains it and calls `process`.
5. The owner calls `stop`. The volatile write `stopping=true` becomes visible to later volatile reads, and `interrupt` wakes an interruptible poll promptly.
6. `poll` throws `InterruptedException`, clearing status. The catch restores it because this boundary is terminating rather than propagating.
7. `finally` releases resources. `runLoop` returns, and the thread becomes `TERMINATED`.
8. Successful `join` lets the owner observe all worker actions. If timeout expires first, `isAlive()` causes a visible shutdown failure.

There is a deliberate policy question: queued jobs may remain unprocessed. A drain-before-stop service would reject new jobs, process the existing queue, then interrupt only after a grace deadline. Cancellation semantics must be documented rather than accidental.

Complexity and performance

Thread creation and platform-thread stacks consume nontrivial native/runtime resources. Starting one platform thread per tiny task can spend more time scheduling than executing. A blocking queue offers $O(1)$ expected enqueue/dequeue for typical linked implementations, but contention and wakeups dominate constants.

Polling every second is not needed for visibility because `interrupt` wakes the wait; an untimed `take()` could reduce periodic wakeups. The timed poll in the example also provides a natural health/cancellation checkpoint. A busy loop would offer lower theoretical response latency but waste CPU.

Cancellation check frequency trades overhead for response time. Check between bounded chunks, not once per trivial arithmetic operation and not only after an unbounded task. Blocking operations should have deadlines when external systems can hang independently of interruption.

> **HotSpot note:** Platform-thread stack reservation, native mapping, scheduling transitions, safepoint interaction, and diagnostic state details are HotSpot/OS dependent. Thread priorities are hints mapped differently across operating systems and should not enforce correctness.

WATCH OUT | Edge cases and common mistakes

- Calling `run()` when concurrency was intended.
- Starting a thread in a constructor, allowing `this` to escape before construction completes.
- Reusing a terminated `Thread` object.
- Assuming `interrupt()` kills the target or closes every I/O operation.
- Catching `InterruptedException`, logging it, and continuing.
- Restoring interrupt status but continuing an interruptible call in a tight loop.
- Using `Thread.stop`, `suspend`, or `resume`; these unsafe/deprecated mechanisms can expose broken invariants or deadlock.
- Depending on thread state polling for coordination.
- Marking critical workers daemon and expecting guaranteed cleanup.
- Waiting forever during shutdown without a deadline or diagnostics.
- Treating an uncaught-exception log as sufficient supervision.

`sleep` does not release monitors a thread owns. `wait` releases the particular monitor as part of its condition-wait protocol. Confusing them can create long lock stalls.

Production engineering notes

Every thread needs an owner, name, failure policy, cancellation mechanism, and termination observation. Prefer executors or structured task scopes where available over scattered raw threads, but the same ownership questions remain. Name threads with subsystem purpose rather than per-request secrets.

A shutdown sequence should normally:

1. Mark the component not ready and stop admitting work.
2. Request cooperative cancellation or graceful draining.
3. Close resources that unblock non-interruptible operations.
4. Wait with a deadline derived from the platform's termination grace period.
5. Capture thread stacks and report remaining work if the deadline expires.
6. Let process-level orchestration escalate only after evidence and policy.

Production incident example: a consumer deployment takes the full 30-second grace period. A thread dump shows the worker blocked in `BlockingQueue.take`; shutdown set a plain `boolean` but never interrupted. Because no item arrives, the loop cannot recheck the flag. Fix the protocol by safe publication plus interrupt/queue signal, then join and expose shutdown duration.

Do not use interruption as a generic business error. Preserve its meaning as cancellation. If a library translates it, include the original cause and restore status unless the API contract has transferred cancellation through another explicit result.

TRY IT | Interview questions and model answers

What is the difference between `start()` and `run()`?

`start()` arranges a new thread of execution that invokes `run` once and creates a happens-before edge from earlier actions. Calling `run()` directly is a normal synchronous call on the current thread.

How does interruption work?

It is a cooperative request represented by interrupt status. Interruptible blocking APIs often throw `InterruptedException` and clear status. Code should propagate the exception or restore status and exit at a boundary. It cannot safely force arbitrary code to stop.

Why restore interrupt status?

If a method cannot propagate `InterruptedException`, restoring preserves the cancellation signal for higher-level code. Restoration should be paired with return, termination, or another explicit policy, not swallowed by continued work.

What does `join()` guarantee?

It waits for termination. When it returns successfully because the target terminated, all actions in that thread happen-before actions after the join in the waiting thread. A timed join can return while the target remains alive.

Daemon versus non-daemon?

Non-daemon threads keep the JVM alive. The JVM may exit when only daemon threads remain, without guaranteeing daemon cleanup. Daemon status is therefore unsuitable as the sole lifecycle policy for critical work.

TRY IT | Exercises

1. Change the worker to graceful drain semantics and define what happens at the deadline.
2. Write a CPU-bound search that responds to interruption every bounded chunk.
3. Demonstrate `Thread.interrupted()` clearing status and `isInterrupted()` preserving it.
4. Add an uncaught-exception handler that marks component health unhealthy.
5. Diagnose a shutdown where a worker is blocked in socket I/O that ignores interruption.
6. Explain the happens-before paths created by `start`, queue handoff, volatile stop, and `join`.

RECAP | Chapter summary

Java threads move through diagnostic lifecycle states, but correctness comes from explicit synchronization and ownership, not state polling. Interruption is a cooperative cancellation request. Interruptible waits convert it to `InterruptedException`, which must be propagated or restored with a deliberate exit policy. Reliable components stop admission, request cancellation, unblock waits, release resources, join with a deadline, and report nontermination.

TRY IT | Revision checklist

- [ ] I can explain all `Thread.State` values without treating them as scheduler guarantees.
- [ ] I know why `start` differs from `run` and cannot be repeated.
- [ ] I can propagate or restore interruption correctly.
- [ ] I can identify APIs that may require cancellation beyond interruption.
- [ ] I can design owned startup and bounded shutdown.
- [ ] I understand daemon and uncaught-exception risks.
- [ ] I can state the `start` and `join` happens-before rules.

SDE-2 CORE CHAPTER

Chapter 3 - Synchronization, Intrinsic Locks, and Explicit Locks

Learning objectives

- Explain mutual exclusion and memory visibility provided by Java monitors.
- Choose the correct lock object and scope for a shared invariant.
- Use `wait`, `notifyAll`, `ReentrantLock`, and `Condition` with condition predicates.
- Handle interruption and exceptions without leaking locks.
- Compare intrinsic locks, explicit locks, read-write strategies, and lock-free alternatives.

WHY IT WORKS | Why this matters at SDE-2

Most lock bugs are design bugs, not syntax bugs. Synchronizing a setter while leaving a reader unguarded does not protect an invariant. Calling external code while holding a lock can turn one slow dependency into global request serialization. A missed signal or `if` around `wait` can hang only under rare timing. An explicit lock without `finally` can permanently block a subsystem after one exception.

An SDE-2 should state what data a lock guards, which operations must be atomic together, how waiting threads are signaled, and how shutdown interrupts those waits. The goal is not "make the method thread-safe" but preserve a named invariant under all interleavings.

WHY IT WORKS | First-principles model

A lock establishes ownership of a critical section. Mutual exclusion ensures at most one holder executes code guarded by that lock. The Java Memory Model also orders state:

TEXT

```
Thread A                              Thread B
lock L                               lock L (later)
  mutate guarded state
unlock L -------- happens-before ---> read guarded state
```

Condition waiting adds another idea. A thread cannot make progress until a predicate over guarded state becomes true. It must atomically release the lock and wait, then reacquire before rechecking:

```text
lock
while (!predicate) {
    await: release lock + wait + reacquire lock
}
change guarded state
unlock
```

Signals are hints that state may have changed. The predicate, not the notification, determines permission to proceed.

Core terminology

- **Monitor/intrinsic lock:** Lock and wait-set behavior associated with every Java object.
- **Critical section:** Code that accesses a shared invariant under its guarding lock.
- **Reentrancy:** A thread holding a lock can acquire it again, with a hold count.
- **Contention:** Multiple threads compete for the same lock.
- **Condition predicate:** Boolean expression over guarded state required for progress.
- **Wait set:** Threads waiting through `Object.wait` on a monitor.
- **Condition:** Explicit wait queue associated with a `Lock`.
- **Fair lock:** Lock configured to favor longer-waiting contenders under documented limitations.
- **Read-write lock:** Lock with shared read and exclusive write modes.
- **Optimistic read:** Read attempted without exclusive ownership, followed by validation.

Detailed mechanics

Every object can serve as a monitor. Entering a `synchronized (lock)` statement acquires that monitor; normal or abrupt exit releases it. An instance `synchronized` method locks `this`. A static synchronized method locks the corresponding `Class` object. These are different locks, so mixing instance and static synchronization does not serialize access unless that is the intended design.

Intrinsic locks are reentrant. If method A holds `this` and calls synchronized method B on the same object, the thread proceeds and increments its logical hold count. It must exit the matching number of acquisitions before another thread can enter.

Monitor exit happens-before a later monitor entry on the same object. This supplies visibility for all earlier actions, not just fields declared inside the synchronized block. The discipline still requires every conflicting access to participate through the same lock or another valid ordering mechanism.

`Object.wait()` requires the caller to own that object's monitor. It releases that monitor and waits; upon wakeup it must reacquire before returning. It can return due to notification, interruption, timeout, or spurious wakeup. Therefore test the predicate in a loop. `notify()` wakes one arbitrary waiter; using it when different

predicates share a wait set can wake an ineligible thread and strand the eligible one. `notifyAll()` is safer for correctness, though potentially more expensive.

`sleep()` does not release owned monitors. `yield()` is only a scheduling hint. Neither creates a happens-before relationship suitable for publication.

`ReentrantLock` provides monitor-like exclusion and JMM effects with additional operations:

- `tryLock()` avoids indefinite acquisition and supports timed attempts.
- `lockInterruptibly()` lets cancellation abort lock acquisition.
- multiple `Condition` instances separate wait sets for different predicates;
- optional fairness influences admission policy;
- lock state and queue inspection support diagnostics, with snapshot limitations.

Always unlock in `finally` after a successful acquisition:

JAVA

```
lock.lock();
try {
    changeState();
} finally {
    lock.unlock();
}
```

Do not place `lock()` inside the `try` if acquisition can fail/interrupt and code might then unlock a lock it never acquired. With timed `tryLock`, branch on the boolean.

A `Condition` belongs to one lock. `await` releases that lock and reacquires before returning, with interruptible, uninterruptible, and timed variants. `signal`/`signalAll` require lock ownership. As with monitors, use predicate loops.

`ReentrantReadWriteLock` can improve throughput when read sections are long, writes rare, and contention material. For tiny state, bookkeeping and writer starvation concerns can make a normal lock faster. Upgrade from read to write is hazardous; release/read then acquire/write and revalidate, or use a documented conversion mechanism.

`StampedLock` offers write, read, and optimistic-read modes but is not reentrant. Optimistic reads must validate and must not expose inconsistent intermediate data before validation. It has different interruption and ownership semantics from `Lock`, so it is an advanced measured optimization, not a default.

> **Specification boundary:** Java APIs and the JMM define monitor/lock exclusion, waiting contracts, and memory-consistency effects. They do not promise fair scheduling for intrinsic locks, a particular queue algorithm, adaptive spinning, or an exact mapping to OS mutexes.

TRACE IT | Worked Java example

JAVA (continued 1/2)

```
import java.util.ArrayDeque;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.ReentrantLock;

public final class BoundedBuffer<E> {
    private final ArrayDeque<E> elements = new ArrayDeque<>();
    private final int capacity;
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition notEmpty = lock.newCondition();
    private final Condition notFull = lock.newCondition();

    public BoundedBuffer(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException("capacity");
        this.capacity = capacity;
    }

    public void put(E element) throws InterruptedException {
        if (element == null) throw new NullPointerException("element");
        lock.lockInterruptibly();
        try {
            while (elements.size() == capacity) {
                notFull.await();
            }
            elements.addLast(element);
            notEmpty.signal();
        } finally {
            lock.unlock();
```

JAVA (continued 2/2)

```
        }
    }

    public E take() throws InterruptedException {
        lock.lockInterruptibly();
        try {
            while (elements.isEmpty()) {
                notEmpty.await();
            }
            E result = elements.removeFirst();
            notFull.signal();
            return result;
        } finally {
            lock.unlock();
        }
    }

    public int size() {
        lock.lock();
        try {
            return elements.size();
        } finally {
            lock.unlock();
        }
    }
}
```

Two conditions avoid waking producers when only consumers can proceed and vice versa. The queue, its size, and capacity invariant are always accessed under one lock.

TRACE IT | Execution or memory walkthrough

Assume capacity one and an empty buffer:

1. Consumer C calls `take`, acquires `lock`, observes empty, and calls `notEmpty.await`.
2. `await` atomically places C in the condition wait protocol and releases `lock`. C is not consuming CPU.
3. Producer P acquires `lock`, observes space, appends A, and calls `notEmpty.signal`.
4. Signal makes one waiter eligible to transfer/reacquire; it does not hand the lock directly to C. P still owns the lock.
5. P exits `finally` and unlocks. Its state changes happen-before C's successful later acquisition through lock semantics.
6. C reacquires, returns from `await`, and rechecks `elements.isEmpty()` in the loop. It removes A and signals `notFull`.
7. If another consumer acquired first and removed A, C would find empty again and await. This is why `if` is incorrect even without a spurious wakeup.

If C is interrupted during `await`, the call throws after reacquiring/processing according to the condition contract. `finally` releases the lock, and `InterruptedException` propagates. The queue invariant remains intact.

The memory shape is simple: the `BoundedBuffer` owns one deque and lock. Waiting threads are managed through condition/lock implementation nodes outside the domain deque; they do not need application-level polling flags.

Complexity and performance

Deque insertion/removal is amortized $O(1)$; lock acquisition is $O(1)$ algorithmically but can wait without a finite bound under contention. A condition wait releases CPU resources but park/unpark and scheduling add latency. `size` is exact at the linearization point under the lock and can be stale immediately after return.

Lock performance depends on critical-section duration, arrival rate, core count, preemption, and fairness. The utilization principle matters: as time inside a serialized region approaches capacity, queueing delay rises sharply. Reduce shared state and critical-section work before replacing the lock primitive.

Measure both acquisition wait and hold duration; either can dominate tail latency.

Fair `ReentrantLock` can reduce barging/starvation in some workloads but often lowers throughput and does not guarantee fair thread scheduling. `tryLock()` can barge even on a fair lock under documented behavior. Measure the workload whose property matters.

> **HotSpot note:** HotSpot has evolved monitor representations, fast paths, spinning, and inflation/deflation policies across JDK releases. Source-level performance should not rely on a particular object-header lock encoding. Uncontended `synchronized` can be highly optimized.

WATCH OUT | Edge cases and common mistakes

- Locking only writes while ordinary reads race.
- Guarding one invariant with multiple unrelated lock objects.
- Synchronizing on an externally accessible string, boxed value, collection, or replaceable field.
- Calling `wait`, `notify`, or `notifyAll` without owning the same object's monitor.
- Using `if` instead of `while` around a condition wait.
- Assuming a signal transfers lock ownership or proves the predicate true.
- Forgetting `unlock` in `finally`.
- Calling slow network/database/user callbacks while holding a shared lock.
- Returning a mutable guarded collection reference to callers.
- Choosing a fair/read-write/stamped lock without measured need.
- Acquiring locks in inconsistent order; analyzed deeply in Master Chapter 38.

An intrinsic lock acquisition cannot be interrupted or timed. If bounded lock wait is a requirement, an explicit lock may be necessary. Conversely, replacing a simple `synchronized` block with `ReentrantLock` only for perceived speed adds failure modes without guaranteed gain.

Production engineering notes

Document each invariant and its guard:

```java
// Guarded by lock: elements.size() <= capacity and all deque access.
```

Keep critical sections small in elapsed time, not merely line count. Copy needed state under lock, release, then perform logging, serialization, callbacks, or remote I/O. If the operation must be atomic with an external system, a Java lock alone cannot provide distributed atomicity; use transactions, idempotency, or a workflow protocol.

Incident example: request p99 jumps from 50 ms to 8 seconds while CPU is low. Thread dumps show 180 threads `BLOCKED` entering a synchronized cache method; the owner is performing a remote refresh inside the monitor. One slow upstream serialized the service. Redesign with an immutable snapshot, single-flight refresh outside the publication lock, a timeout, and stale-value policy.

Thread dumps show monitor owner and contenders for intrinsic locks and often explicit-lock parking stacks. JFR lock events and profiles reveal duration and hot call sites. One dump is a snapshot; collect a short series and correlate with request/CPU timelines.

For shutdown, waiting methods should propagate interruption. A condition-based component can also set a closed predicate under lock and `signalAll` so every waiter can distinguish closure from temporary empty/full state.

TRY IT | Interview questions and model answers

What does `synchronized` guarantee?

Mutual exclusion for code using the same monitor and memory ordering: an unlock happens-before a subsequent lock of that monitor. It is reentrant and releases on normal or abrupt block exit. It does not make unrelated unsynchronized accesses safe.

Why call `wait` in a loop?

Wakeups can be spurious, a notification can target a thread whose predicate is false, or another thread can consume the state before this waiter reacquires the lock. The condition predicate must be rechecked while holding the lock.

When would you use `ReentrantLock` instead of `synchronized`?

When requirements include timed or interruptible acquisition, multiple conditions, or a deliberately evaluated fairness/inspection feature. For simple scoped exclusion, `synchronized` is safer and concise.

Does `notify` wake the most appropriate waiter?

No such selection is guaranteed. With multiple predicates on one monitor, `notify` can wake an ineligible waiter. `notifyAll` plus predicate loops is safer, or separate explicit `Condition` queues.

Read-write lock versus normal lock?

A read-write lock can help with long, frequent reads and rare writes under real contention. It adds coordination, can complicate upgrades, and may reduce performance for short sections. Benchmark the actual read/write mix.

TRY IT | Exercises

1. Add `close()` to `BoundedBuffer` so all waiting producers/consumers terminate with defined behavior.
2. Implement the same buffer with `synchronized`, `wait`, and `notifyAll`.
3. Demonstrate why replacing each `while` with `if` is incorrect using a two-consumer trace.
4. Refactor code that performs a callback under lock into snapshot-then-callback form.
5. Compare `lock`, `lockInterruptibly`, and timed `tryLock` cancellation behavior.
6. Write the invariant and lock-order documentation for a two-account transfer.

RECAP | Chapter summary

Locks protect named invariants by combining mutual exclusion with happens-before ordering. Intrinsic monitors are reentrant and support one wait set; explicit locks add timed/interruptible acquisition and multiple conditions. Condition waits always require a predicate loop, and every successful explicit acquisition requires `finally`-based release. Performance comes mainly from reducing contention and critical-section duration, not selecting a fashionable lock.

TRY IT | Revision checklist

- [ ] I can identify the exact invariant and lock object.
- [ ] I can explain monitor unlock-to-lock visibility.
- [ ] I know why wait/await uses a loop and reacquires the lock.
- [ ] I can use `ReentrantLock` and `Condition` exception-safely.
- [ ] I can choose among intrinsic, explicit, read-write, and optimistic locking.
- [ ] I avoid external calls and mutable-state escape under a shared lock.
- [ ] I can investigate contention using dumps, JFR, and a timeline.

SDE-2 CORE CHAPTER

Chapter 4 - Volatile, Atomics, CAS, and Happens-Before in Practice

Learning objectives

- Prove publication and visibility using happens-before rather than timing intuition.
- Identify when volatile is sufficient and when an invariant needs an atomic transition or lock.
- Explain compare-and-set, retry loops, linearization points, and progress properties.
- Recognize ABA and choose versioning, stamped references, or a different design.
- Use atomic counters, immutable state snapshots, and contention-aware accumulators correctly.

WHY IT WORKS | Why this matters at SDE-2

Lock-free code can be wrong while looking sophisticated. Two atomic fields do not make a two-field invariant atomic. A volatile flag can publish prior writes but cannot prevent duplicate check-then-act. A CAS loop can burn a core under contention. A `LongAdder` can improve metrics throughput but is unsuitable for an exact bank balance or sequence number.

SDE-2 interviews expect both correctness and judgment. You should locate the linearization point, name the happens-before edge, state whether the operation is lock-free or wait-free, and explain why a simpler lock might be better. Production design additionally needs overflow, observability, retry, and shutdown behavior.

WHY IT WORKS | First-principles model

Shared-state correctness has two layers:

1. **Ordering/visibility:** Which writes must a reader observe? Use happens-before edges.
2. **Atomic state transition:** Which read-decision-write sequence must appear indivisible? Use a lock, CAS, or a higher-level concurrent operation.

Volatile solves the first layer for a variable and surrounding actions:

TEXT

```
writer                                  reader
snapshot = fullyBuilt;
volatile current = snapshot; --HB--> read volatile current
                                        use fully initialized snapshot
```

CAS can solve a single-location transition:

```text
loop:
  observed = state
  proposed = f(observed)
  if compareAndSet(observed, proposed): success
  else: another transition won; retry from new state
```

The successful CAS is the linearization point: the instant the operation appears to take effect.

Core terminology

- **Volatile variable:** Field whose accesses participate in the JMM volatile synchronization order and visibility rules.
- **Atomic variable:** Library wrapper such as `AtomicInteger` or `AtomicReference` supporting indivisible operations on one location.
- **CAS:** Compare-and-set; update only if the current value equals an expected value under the operation's comparison semantics.
- **Linearization point:** Single conceptual instant at which a concurrent operation takes effect.
- **Lock-free:** System-wide progress is guaranteed under continued scheduling, though one thread may starve.
- **Wait-free:** Each operation completes in a bounded number of its own steps.
- **Obstruction-free:** A thread completes if it eventually runs alone long enough.
- **ABA problem:** A location changes A to B and back to A, making equality alone hide intervening history.
- **Contention:** Concurrent attempts compete to update the same state.
- **False sharing:** Independent hot values trigger cache-coherence traffic because of physical proximity.

Detailed mechanics

A volatile write happens-before every subsequent read of that volatile in synchronization order. Program order and transitivity publish ordinary writes before the volatile write to code after the volatile read. Volatile access is individually atomic, including volatile `long` and `double`.

Volatile does not combine operations. `volatile int count; count++;` remains a read, calculation, and write. Two threads can read 5 and both write 6. Similarly, this is unsafe:

```java
if (!initialized) {
    initialized = true;
    initialize();
}
```

Even if `initialized` is volatile, two threads can both read false before either writes true, and setting it before initialization can expose the wrong protocol. Use class initialization, a lock, a future, or correctly designed CAS state.

Volatile is a good match for an independent cancellation flag, one-writer status, or reference to an immutable snapshot. Updating an immutable `Config` and publishing its single reference makes all fields move together. Publishing separate volatile host and port fields permits mixed-version snapshots.

Atomic classes offer operations such as `getAndIncrement`, `compareAndSet`, `getAndUpdate`, and `accumulateAndGet`. These are more than volatile wrappers because read-modify-write is indivisible for that atomic location. Atomic object methods have documented memory effects broadly aligned with their access modes. Newer APIs such as `VarHandle` expose a range from plain/opaque/acquire/release to volatile access; use weaker modes only with a written proof and measured need.

A CAS loop must recompute from the newly observed value after failure. The update function can execute multiple times, so it must be side-effect free. Do not charge a card, append to an external log, or increment a separate metric inside a function passed to `updateAndGet`; retries can duplicate side effects.

CAS is optimistic. Under low contention it avoids parking and often performs well. Under high contention many threads calculate losing updates, retry, and generate cache-line traffic. Lock-free means some operation makes progress; it does not mean every caller has bounded latency or that no scheduling support is involved.

ABA matters when equality is used as evidence that no relevant change occurred. In a lock-free stack, thread T1 reads head A and next C, pauses, while T2 pops A, changes the stack, and later pushes the same A object back. T1's CAS from A to C can succeed even though history changed and C may no longer represent the intended successor. Solutions include immutable/non-reused nodes under GC-specific reasoning, version counters, `AtomicStampedReference`, `AtomicMarkableReference`, hazard/epoch schemes in native memory, or a lock.

Java garbage collection prevents a freed Java object address from being manually reallocated behind a live reference in ordinary code, removing one classic native ABA route. It does not eliminate logical ABA when the same reference/value is deliberately restored.

`AtomicStampedReference` atomically compares reference plus integer stamp. Stamp overflow is theoretically possible, so the stamp width and operation lifetime matter. A monotonic version embedded in an immutable state object often makes domain history clearer.

`LongAdder` distributes updates across internal cells under contention, then sums them. It is excellent for high-rate statistics where a momentarily non-atomic aggregate is acceptable. `sum()` is not an atomic snapshot relative to concurrent updates, and `sumThenReset()` can be inappropriate when exact accounting is required. Use `AtomicLong` or locking for exact sequences/balances.

> **Specification boundary:** The JMM and `java.util.concurrent.atomic` APIs define visibility and atomic-operation contracts. They do not require one CPU instruction, a particular cache protocol, wait-free progress for all atomic methods, or a fixed physical layout.

TRACE IT | Worked Java example

JAVA

```java
import java.util.concurrent.atomic.AtomicReference;

public final class Inventory {
    private record State(int available, long version) {
        State {
            if (available < 0) throw new IllegalArgumentException("negative");
        }
    }

    private final AtomicReference<State> state;

    public Inventory(int initial) {
        state = new AtomicReference<>(new State(initial, 0));
    }

    public boolean reserve(int units) {
        if (units <= 0) throw new IllegalArgumentException("units");
        while (true) {
            State observed = state.get();
            if (observed.available() < units) return false;
            State proposed = new State(
                    observed.available() - units,
                    observed.version() + 1);
            if (state.compareAndSet(observed, proposed)) return true;
            Thread.onSpinWait();
        }
    }

    public int available() {
        return state.get().available();
    }
}
```

Availability and version move in one immutable object referenced by one atomic location. No observer can see new availability with an old version. The successful CAS linearizes each reservation.

This in-memory reservation is not a substitute for a database transaction across processes. Its scope is one JVM object instance.

TRACE IT | Execution or memory walkthrough

Start with `State(5, 0)`. Threads A and B both call `reserve(4)`:

1. A reads reference S0 -> `(5,0)`.
2. B reads the same reference S0.
3. A allocates proposed S1 -> `(1,1)`.
4. B allocates proposed S2 -> `(1,1)`; equal content does not make it the same object reference.
5. A performs CAS expected S0, update S1. It succeeds and atomically changes the location.
6. B performs CAS expected S0, update S2. Current reference is S1, so it fails.
7. B retries and reads S1. Available 1 is less than 4, so B returns false.

Exactly one reservation succeeds; availability never becomes negative. A lock-free system-progress claim applies because a failed CAS indicates another update won. B itself was not guaranteed to win.

The immutable objects are ordinary heap objects. Failed proposed states become unreachable and add allocation/GC pressure. A packed `AtomicLong` could encode fields without allocation, but packing introduces bit widths, overflow, readability, and migration constraints. Optimize only after evidence.

The atomic read that observes S1 also gives the documented visibility needed to read that fully constructed record. Final record fields further support immutable construction, but publication still uses the atomic reference.

Complexity and performance

In the absence of contention, `reserve` is O(1) time and allocates one small state. Under contention, one call can retry an unbounded number of times, so it is not wait-free. Total useful updates still progress if threads continue running and the atomic implementation provides the expected progress behavior.

CAS loops work best for small, fast, independent transformations. If validation is expensive, contention high, or the invariant spans several mutable structures, a lock can reduce wasted work and produce better tail latency. Backoff may reduce contention but complicates fairness and tuning.

Atomics create hot memory locations. An `AtomicLong` incremented by every request can become a coherence bottleneck. `LongAdder` trades exact instantaneous state and memory for scalable distributed updates. Sharding domain state can also improve locality, but requires aggregation semantics.

`Thread.onSpinWait()` is only a performance hint and does not add ordering or guarantee progress. A long spin should yield to a blocking design or explicit backoff to avoid wasting CPU.

> **HotSpot note:** HotSpot intrinsifies many atomic operations into CPU-specific instructions and barriers when possible. CAS instruction costs, spurious failure behavior of weak variants, padding, and fence mappings differ across x86-64, AArch64, and JVM versions.

WATCH OUT | Edge cases and common mistakes

- Believing volatile makes `++`, check-then-act, or multi-field invariants atomic.
- Updating two atomics separately and calling the pair transactionally consistent.
- Performing side effects inside a retryable atomic update function.
- Ignoring counter overflow or stamp wraparound.
- Treating `LongAdder.sum()` as an exact linearizable balance.
- Assuming lock-free means starvation-free, bounded latency, or always faster.
- Retrying CAS without rereading and recomputing proposed state.
- Using reference CAS when logical equality, not identity, was intended, or vice versa.
- Dismissing ABA merely because Java has GC.
- Publishing a mutable object through volatile and then mutating it without synchronization.
- Using several volatile fields when one immutable snapshot is the actual invariant.

A volatile collection reference does not make collection operations thread-safe. Volatile orders replacement of the reference; concurrent mutation still requires a concurrent collection or other synchronization.

Production engineering notes

Write down each operation's linearization point and scope. `Inventory.reserve` is atomic only inside one process and instance; a replicated service requires a database conditional update, consensus-backed store, partition ownership, or idempotent distributed workflow.

Monitor CAS failure/retry rates, CPU, allocation, and p99 latency. A regression after traffic growth may be coherence contention rather than application computation. Profiles can show atomic intrinsics and spin hot spots; JFR/OS tools can correlate CPU saturation.

Incident example: a metrics endpoint occasionally reports fewer completed requests than the previous scrape. The code uses `LongAdder.sumThenReset()` while updates continue, assuming an exact interval transfer. Some updates race with cell summation/reset. Fix by accepting approximate monotonic cumulative counters, using a metrics library's supported model, or serializing exact interval accounting if the business truly requires it.

For configuration, construct an immutable snapshot, validate it completely, then publish one volatile/atomic reference. Keep the previous snapshot on validation failure. Readers perform one read and use that local snapshot for the whole operation, avoiding mixed versions if a refresh occurs midway.

TRY IT | Interview questions and model answers

What does volatile provide?

Volatile access is atomic and ordered in the synchronization order. A volatile write happens-before subsequent reads of that variable, publishing prior actions to code after the read. It does not make compound operations or a multi-variable invariant atomic.

How does CAS work?

It atomically compares the current value with an expected value and installs an update only on match. A CAS loop reads state, computes a side-effect-free proposal, attempts CAS, and recomputes after failure. The successful CAS is the linearization point.

What does lock-free mean?

Under continued execution, the system as a whole makes progress; individual threads may repeatedly lose and starve. It does not mean no coordination, no blocking anywhere in the runtime, or better latency under all contention.

Explain ABA.

A location changes from A to B and back to A, so a comparison with A succeeds despite relevant intermediate history. Use a version/stamp, a design that prevents reuse/restoration, a safe reclamation scheme, or a lock when history matters.

AtomicLong versus LongAdder?

`AtomicLong` gives linearizable exact single-location updates and reads. `LongAdder` spreads writes for throughput but its aggregate is not one atomic snapshot. Use it for statistics, not exact balances or IDs.

TRY IT | Exercises

1. Add `restock` to `Inventory`, including overflow handling and a linearization-point explanation.
2. Implement the inventory with `ReentrantLock` and compare correctness proof and contention behavior.
3. Show a two-atomic invariant that produces a mixed snapshot; replace it with one immutable atomic state.
4. Construct a logical ABA trace for a lock-free stack and add a stamp.
5. Benchmark `AtomicLong` and `LongAdder` across thread counts, then state what correctness each benchmark ignores.
6. Review an `updateAndGet` lambda containing a log or external call and redesign it.

RECAP | Chapter summary

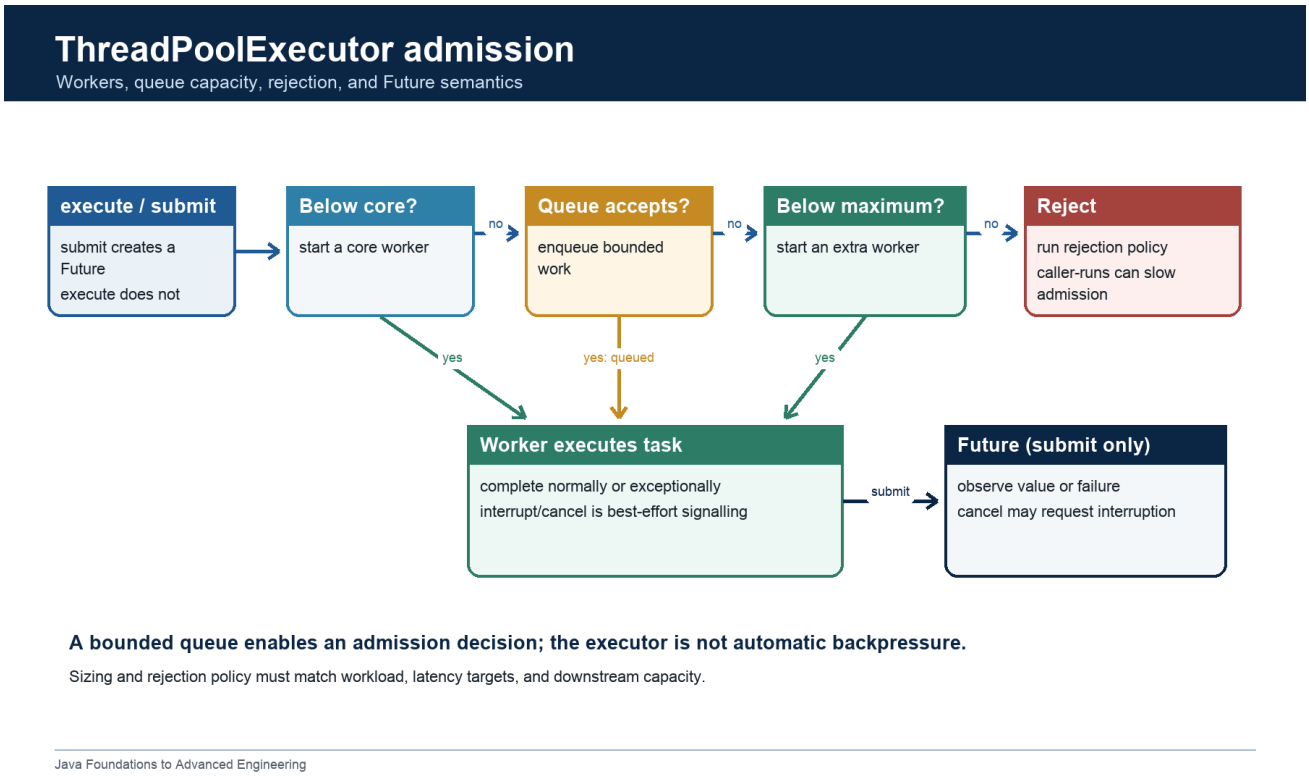
Volatile supplies ordered visibility, not compound atomicity. Atomic classes make one-location transitions linearizable, and CAS loops turn failed competition into retry. Correct lock-free design requires side-effect-free proposals, explicit progress expectations, ABA analysis, overflow handling, and contention measurement. Immutable snapshots published through one volatile or atomic reference often provide the clearest practical design.

TRY IT | Revision checklist

- [ ] I can prove volatile publication using happens-before.
- [ ] I know which invariants volatile cannot protect.
- [ ] I can identify a CAS loop's linearization point.
- [ ] I distinguish lock-free, wait-free, and starvation-free behavior.
- [ ] I can explain logical ABA and versioned solutions.
- [ ] I choose `AtomicLong` versus `LongAdder` by correctness needs.
- [ ] I avoid side effects in retryable update functions.

SDE-2 CORE CHAPTER

Chapter 5 - Executors, Futures, CompletableFuture, and Work Scheduling



ThreadPoolExecutor admission, worker, rejection, and Future semantics

Learning objectives

- Separate task submission, execution policy, completion, and shutdown.
- Configure thread pools with intentional sizing, bounded queues, and saturation behavior.
- Use `Future` timeouts and cancellation without confusing them with task termination.
- Compose `CompletableFuture` stages and handle failures precisely.
- Avoid common-pool blocking, unbounded admission, hidden exceptions, and orphaned work.

WHY IT WORKS | Why this matters at SDE-2

Executors turn a concurrency problem into a capacity system. An unbounded queue can preserve acceptance while latency and memory grow until failure. An oversized blocking pool can overwhelm a database. A `Future.get` timeout can return while the task continues consuming resources. A `CompletableFuture` callback can run on a request thread, pool worker, or common pool depending on completion timing and method choice.

An SDE-2 must design ownership and overload behavior, not just call `submit`. Interviews expect differences among `execute`, `submit`, `thenApply`, `thenCompose`, `handle`, and `exceptionally`. Production expects queue depth, rejection, task age, and shutdown behavior to be observable.

WHY IT WORKS | First-principles model

An executor separates **what** should run from **where and when** it runs:

TEXT

```
producer -> admission -> work queue -> workers -> completion
                |                          |
            reject/backpressure       result/failure
```

Capacity is finite even when a data structure is unbounded. If arrival rate exceeds service rate for long enough, queued work, latency, and memory grow. A correct policy bounds one or more of: accepted concurrency, queue length, wait time, work per request, or upstream arrival.

A completion object represents a state machine:

TEXT

```
incomplete -> completed normally(value)
           -> completed exceptionally(cause)
           -> cancelled(cancellation outcome)
```

Composition transforms state without blocking a worker merely to wait for another stage.

Core terminology

- **Executor:** Interface accepting tasks for execution.
- **ExecutorService:** Executor with lifecycle, submission, futures, and shutdown operations.
- **ThreadPoolExecutor:** Configurable platform-thread pool with core/max sizes, queue, factory, and rejection policy.
- **Future:** Handle for completion, result retrieval, cancellation request, and status.
- **Saturation policy:** Behavior when an executor cannot accept more work.
- **Backpressure:** Propagating limited capacity to producers rather than buffering without bound.
- **Work stealing:** Workers take tasks from other workers' queues to balance computation.
- **Completion stage:** Dependent asynchronous computation triggered by another completion.
- **Fan-out/fan-in:** Start multiple tasks and combine their completions.
- **Deadline:** Absolute remaining-time budget propagated across operations.

Detailed mechanics

`execute(Runnable)` submits work with no result handle. If the task throws, uncaught-exception behavior may expose it through the worker/thread mechanism. `submit` returns a `Future`; thrown exceptions are captured and later wrapped by `Future.get` in `ExecutionException`. Ignoring the returned future can hide failure. This distinction often explains why changing `execute` to `submit` made logs disappear.

`ThreadPoolExecutor` admission is frequently misunderstood. In the typical policy:

1. If fewer than core threads exist, create one for the task.
2. Otherwise offer to the work queue.
3. If queue offer fails and thread count is below maximum, add a worker.
4. Otherwise reject.

With an unbounded queue, step 2 nearly always succeeds, so `maximumPoolSize` may never affect load handling. A bounded `ArrayBlockingQueue` makes overload explicit. Rejection choices include aborting with `RejectedExecutionException`, running in the caller, discarding, or discarding oldest. Silent discard is dangerous unless loss is deliberate and measured. `CallerRunsPolicy` can slow producers but may also run expensive work on an event loop or latency-sensitive request thread, so it is not universally safe.

Pool sizing begins with workload, not a magic number. For CPU-bound independent work, a starting point near available processors avoids excessive runnable threads. For blocking work, the rough formula `threads = cores / (1 - blockingFraction)` can guide an experiment, but external capacity and memory usually impose stricter limits. A database with 40 connections cannot benefit from 400 concurrent queries. Measure service time, blocking fraction, utilization, queue delay, downstream limits, and tail latency.

A `Future` result becomes available through `get`. Actions in the asynchronous computation happen-before actions after another thread successfully retrieves the result through `Future.get`, per the API

memory-consistency contract. `get(timeout)` bounds how long the caller waits; `TimeoutException` does not cancel the task. `cancel(true)` requests interruption if running, but success means the Future transitioned to cancelled, not proof that arbitrary target code stopped. `cancel(false)` prevents execution when possible or records cancellation without interruption depending on state.

Correct executor shutdown is two-phase. `shutdown()` rejects new tasks and allows accepted work to finish. Await for a bounded period. If time expires, `shutdownNow()` requests interruption and returns tasks that never commenced. Await again and report remaining nontermination. If the shutdown thread is interrupted, request immediate shutdown, restore status, and return/propagate according to the boundary.

Scheduled executors distinguish fixed rate from fixed delay. Fixed-rate scheduling targets a cadence relative to the initial schedule and can run successive executions close together when behind, but does not concurrently overlap executions of the same periodic task through that scheduling contract. Fixed delay waits a delay after one run completes. If a periodic task throws an uncaught exception, subsequent executions are suppressed. Wrap/report failures deliberately.

`CompletableFuture` combines a writable completion with `CompletionStage` transformations:

- `thenApply`: map one result synchronously when the stage completes.
- `thenCompose`: map to another stage and flatten it, avoiding nested futures.
- `thenCombine`: combine two independent successful results.
- `allOf`: complete when all supplied stages complete; it does not collect typed results.
- `exceptionally`: recover from failure with a value.
- `handle`: transform both success and failure into a result.
- `whenComplete`: observe success/failure while generally preserving the original outcome.

Non-`Async` dependent actions can run in the thread that completes the previous stage or the thread attaching the action if already complete. `Async` variants without an executor use the default asynchronous facility, commonly the common `ForkJoinPool`. Supply an explicit executor for isolation, capacity, naming, and blocking behavior.

`thenApply` is wrong when the function returns a future: it creates `CompletableFuture<CompletableFuture<T>>`. Use `thenCompose`. A dependent success stage is skipped if its prerequisite fails. `join()` throws unchecked `CompletionException`; `get()` uses checked `ExecutionException`/`InterruptedException`. Unwrap deliberately without discarding context.

`CompletableFuture.cancel` completes the future exceptionally with `CancellationException`; the `mayInterruptIfRunning` parameter has no interrupt effect for its asynchronous processing according to this class's contract. Cancellation must be designed into the underlying task/resource, not assumed from the graph handle.

> **Specification boundary:** `java.util.concurrent` specifies lifecycle, completion, cancellation requests, and memory-consistency effects. It does not guarantee task start order unless an executor says so, fair worker scheduling, task termination after cancel, or one thread per task.

TRACE IT | Worked Java example

JAVA (continued 1/2)

```
import java.time.Duration;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

public final class PricePipeline implements AutoCloseable {
    private final ThreadPoolExecutor io = new ThreadPoolExecutor(
            8, 8, 0, TimeUnit.MILLISECONDS,
            new ArrayBlockingQueue<>(100),
            new ThreadPoolExecutor.AbortPolicy());

    CompletableFuture<Integer> basePrice(String sku) {
        return CompletableFuture.supplyAsync(() -> loadPrice(sku), io);
    }

    CompletableFuture<Integer> discount(int base) {
        return CompletableFuture.supplyAsync(() -> loadDiscount(base), io);
    }

    CompletableFuture<Integer> finalPrice(String sku) {
        return basePrice(sku)
                .thenCompose(this::discount)
                .thenApply(value -> Math.max(0, value))
                .orTimeout(500, TimeUnit.MILLISECONDS)
                .exceptionally(failure -> 0);
```

JAVA (continued 2/2)

```
    }

    private int loadPrice(String sku) { return 100; }
    private int loadDiscount(int base) { return base - 10; }

    @Override
    public void close() {
        shutdown(io, Duration.ofSeconds(5));
    }

    static void shutdown(ThreadPoolExecutor executor, Duration timeout) {
        executor.shutdown();
        try {
            if (!executor.awaitTermination(timeout.toMillis(), TimeUnit.MILLISECONDS)) {
                executor.shutdownNow();
                if (!executor.awaitTermination(timeout.toMillis(), TimeUnit.MILLISECONDS)) {
                    System.err.println("executor did not terminate");
                }
            }
        } catch (InterruptedException interrupted) {
            executor.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}
```

The bounded queue makes overload observable through rejection. `thenCompose` starts the dependent asynchronous discount only after obtaining the base. The fallback of zero is intentionally visible but would be a

dangerous business default unless the domain defines it; production code should classify timeout, rejection, and data failures separately.

TRACE IT | Execution or memory walkthrough

Successful path:

1. Caller invokes `finalPrice("A").basePrice` submits a task and returns an incomplete future F1.
2. The task enters a worker or bounded queue. If both worker capacity and queue are unavailable, submission throws rejection before a usable pipeline is returned unless handled at that boundary.
3. A worker loads 100 and completes F1. Its completion triggers the `thenCompose` function, which calls `discount(100)` and returns F2.
4. `thenCompose` links rather than nests F2. Another IO task computes 90 and completes F2.
5. `thenApply` maps 90 to 90. The timeout stage races normal completion against its timer semantics; normal completion wins here.
6. `exceptionally` sees no failure and passes the value. The returned future completes with 90.

Failure path: if `loadDiscount` throws, F2 completes exceptionally. `thenApply` is skipped. `orTimeout` does not replace an already completed failure. `exceptionally` receives a completion-wrapped cause and returns 0, converting the final stage to normal success. Metrics and logs must occur before or inside recovery if the failure must remain observable.

Timeout path: the pipeline future completes exceptionally after the configured duration, but underlying I/O may continue. `orTimeout` is not resource cancellation. The I/O client also needs connect/read/request deadlines, and cancellation hooks where supported.

Complexity and performance

Submission is generally O(1), while queueing delay is unbounded in time unless capacity and deadlines constrain it. A queue of capacity `q` uses O(q) references plus retained task graphs. Each task can retain request payloads, traces, and closures, so a "small" queue can hold significant heap.

Little's Law relates average concurrency `L`, arrival rate `lambda`, and average time `W`: `L = lambda * W` in a stable system. If 500 requests/second each occupy an IO operation for 200 ms, average in-flight demand is about 100 before bursts. Downstream limits and desired utilization determine admission below saturation.

Blocking on one future from a task in the same small executor can create thread starvation deadlock: every worker waits for subtasks queued behind them. Compose asynchronously or use separate capacity domains. Work-stealing pools excel at many nonblocking fork/join tasks; unmanaged blocking can reduce parallelism and starve unrelated common-pool users.

WATCH OUT | Edge cases and common mistakes

- Using an unbounded queue and assuming `maximumPoolSize` controls overload.
- Creating an executor per request or forgetting to close an owned executor.
- Ignoring a `Future` returned by `submit`, hiding task exceptions.
- Treating `get(timeout)` as cancellation.
- Assuming `cancel(true)` proves target termination.
- Blocking common-pool workers on slow I/O.
- Using non-`Async` completion actions while assuming a dedicated thread.
- Using `thenApply` for a future-returning function and creating nested futures.
- Recovering every failure to a default and losing incident visibility.
- Scheduling a periodic task whose first exception silently suppresses future runs.
- Letting a queue hold work past caller deadlines.
- Using `CallerRunsPolicy` on an event loop or lock-holding producer without analysis.

Production engineering notes

Treat each executor as a named resource pool with an owner and SLO. Export active workers, pool size, queue size/capacity, oldest task age if feasible, completed count, rejection count, execution duration, queue delay, cancellation, and shutdown duration. Thread names should identify the capacity domain.

Separate pools only when they represent different blocking behavior, priorities, or failure domains; too many pools make total concurrency uncontrollable. The strongest limit is often a semaphore or client pool aligned with a downstream resource, regardless of executor size.

Incident example: heap rises while CPU and database usage remain moderate. The service uses `newFixedThreadPool(32)`, whose factory commonly uses an unbounded queue. A dependency slows, incoming requests continue submitting closures that retain 200 KB payloads, and queue depth reaches 50,000. Fix with a bounded queue, rejection/backpressure, end-to-end deadlines, limited payload retention, and upstream shedding. Increasing heap only delays failure.

Propagate deadlines, not independent full timeouts at every layer. If an incoming request has 800 ms remaining, a downstream stage should not start a new 2-second timeout. When a stage times out, cancel or close the underlying client operation if its API supports it, and ensure late completion cannot commit unwanted side effects.

> **HotSpot note:** The common asynchronous facility normally uses `ForkJoinPool.commonPool()` in mainstream JDKs, with behavior influenced by system properties and environment. Worker implementation, work-stealing queues, compensation for managed blocking, and timer mechanics are library/runtime details, not language guarantees.

TRY IT | Interview questions and model answers

How does `ThreadPoolExecutor` use core size, maximum size, and queue?

It normally creates workers up to core, then queues; only when queueing fails does it grow toward maximum, then reject. Therefore an unbounded queue often makes maximum size irrelevant. The queue and rejection policy are central overload decisions.

How do you size a pool?

Start near processor count for CPU-bound work. For blocking work, estimate blocking fraction and concurrency demand, but cap by downstream capacity, memory, and latency. Validate queue delay, utilization, and tail latency under representative load rather than trust a formula.

`thenApply` versus `thenCompose`?

`thenApply` maps `T` to `U`. If the function maps `T` to `CompletionStage<U>`, use `thenCompose` to flatten the asynchronous dependency rather than produce a nested stage.

How do CompletableFuture exceptions work?

A failed prerequisite skips normal dependent stages. `exceptionally` recovers failures, `handle` maps both outcomes, and `whenComplete` observes while preserving outcome in the ordinary case. `join` reports failure through `CompletionException`; `get` uses `ExecutionException` and is interruptible.

What is correct shutdown?

Stop admission with `shutdown`, await a bounded grace period, request interruption with `shutdownNow` if needed, await again, report survivors, and restore caller interrupt status if shutdown itself is interrupted.

TRY IT | Exercises

1. Add explicit handling for rejection so `finalPrice` returns a failed future rather than throwing synchronously.
2. Replace the default fallback with typed classification for timeout, rejection, and domain failure.
3. Size a pool and queue for a stated arrival rate, service time, downstream connection limit, and burst budget.
4. Create and then fix a thread-starvation deadlock caused by waiting on same-pool subtasks.
5. Compare fixed-rate and fixed-delay execution when one run exceeds the period.
6. Implement fan-out to two independent futures and combine with `thenCombine` under one deadline.

RECAP | Chapter summary

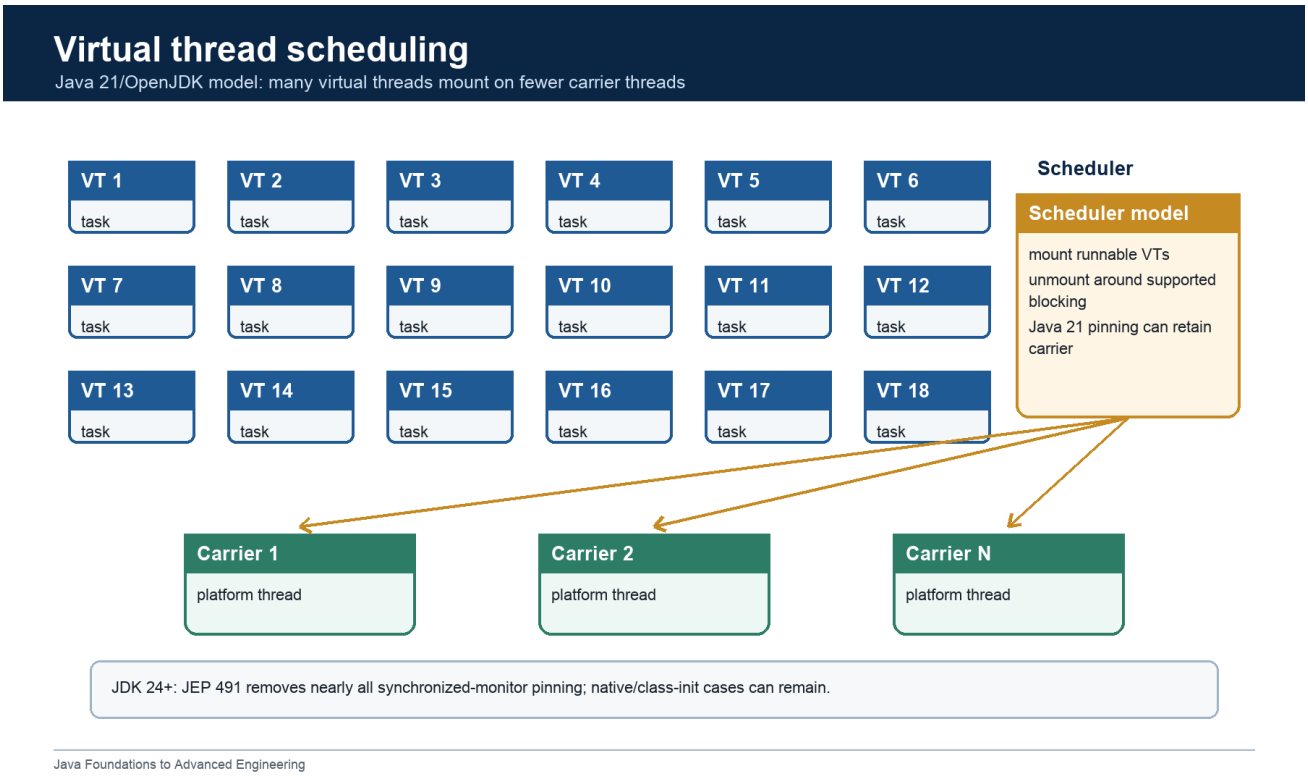
Executors encapsulate scheduling and capacity; futures represent completion, not guaranteed task termination. Bounded queues and explicit rejection make overload controllable. Pool sizing follows computation, blocking, downstream limits, and measured queue latency. CompletableFuture composition avoids blocking when `thenCompose` and combination stages express dependencies, but execution context and exception recovery must be explicit. Every executor needs ownership and bounded shutdown.

TRY IT | Revision checklist

- [ ] I understand executor admission order and unbounded-queue risk.
- [ ] I can size pools from workload and downstream constraints.
- [ ] I know `get(timeout)` and cancellation limitations.
- [ ] I can implement two-phase executor shutdown.
- [ ] I can choose `thenApply`, `thenCompose`, `thenCombine`, and error stages.
- [ ] I specify callback execution context rather than assume it.
- [ ] I can diagnose queue growth, hidden exceptions, and same-pool starvation.

SDE-2 CORE CHAPTER

Chapter 6 - Concurrent Collections and Virtual Threads



Java 21/OpenJDK virtual-thread scheduling with a JDK 24 pinning delta

Learning objectives

- Choose a concurrent collection by access pattern, ordering, consistency, and capacity needs.
- Use atomic map operations instead of unsafe compound check-then-act sequences.
- Explain weakly consistent and snapshot iteration without expecting fail-fast behavior.
- Design Java 21 virtual-thread-per-task services without pooling virtual threads.
- Recognize pinning and bound scarce downstream resources independently of thread count.

WHY IT WORKS | Why this matters at SDE-2

Replacing `HashMap` with `ConcurrentHashMap` prevents structural corruption but does not automatically preserve a business invariant across several calls. An unbounded "concurrent" queue can still exhaust memory. Snapshot iteration can be perfect for configuration listeners and disastrous for a write-heavy list.

Virtual threads change the cost of blocking concurrency, not the capacity of databases, remote APIs, CPU cores, or memory. A service can move from 200 pooled platform threads to 20,000 virtual requests and overload its database in seconds. SDE-2 design therefore separates task representation from resource admission.

WHY IT WORKS | First-principles model

A concurrent collection defines synchronization at the operation boundary. The key question is which compound action is one operation:

TEXT

```
unsafe composition                      atomic collection operation
if (!map.containsKey(k)) {              map.computeIfAbsent(k, loader)
    map.put(k, load(k));
}
```

Thread safety of each call does not make the left pair atomic. Another thread can change the map between calls.

Virtual threads address a different boundary. They let one Java thread represent one task while the runtime schedules many such threads over fewer carrier platform threads:

TEXT

```
virtual tasks:  V1 V2 V3 V4 V5 ... V10000
                  \ | /     \ | /
carrier threads:   P1   P2   P3   ...
                        |
                    CPU cores
```

When supported blocking operations park a virtual thread, its continuation can be unmounted so the carrier runs another. The application keeps straightforward blocking code without dedicating one platform stack/thread to every wait.

Core terminology

- **Linearizable operation:** Appears to take effect at one point between invocation and response.
- **Weakly consistent iterator:** Tolerates concurrent updates, does not throw `ConcurrentModificationException`, and may reflect some updates under its contract.
- **Snapshot iterator:** Iterates an immutable array/version captured when the iterator was created.
- **Blocking queue:** Queue whose operations can wait for capacity or elements.
- **Nonblocking collection:** Collection whose core operations use progress techniques without ordinary mutual-exclusion blocking.
- **Virtual thread:** Lightweight `Thread` implementation finalized in Java 21 for high-throughput blocking tasks.
- **Carrier:** Platform thread on which a virtual thread is mounted for execution.
- **Mount/unmount:** Runtime scheduling of virtual-thread execution state onto/off a carrier.
- **Pinning:** Condition in which a blocked virtual thread cannot unmount from its carrier in the relevant JDK.
- **Bulkhead:** Independent capacity limit preventing one workload/resource from consuming all concurrency.

Detailed mechanics

`ConcurrentHashMap` supports concurrent retrievals and high update concurrency. It rejects null keys and values, which avoids ambiguity between absent and mapped-to-null in concurrent reads. Individual operations such as `putIfAbsent`, conditional `remove`, `replace`, `compute`, `computeIfAbsent`, and `merge` provide atomic compound behavior under their documented contracts.

Use the operation that matches the invariant. A cache loader usually needs `computeIfAbsent`; a state machine may need `compute`; compare-and-remove can use `remove(key, expectedValue)`. Do not split an atomic transition into `get`, decide, `put`. Mapping/remapping functions should be short, nonblocking where possible, and must not recursively update the same map in ways forbidden by the API. Other map keys can change while one key is computed, so map-wide invariants still need another design.

ConcurrentHashMap iterators, spliterators, and enumerations are weakly consistent. They do not freeze the map, do not throw fail-fast exceptions, and may observe updates occurring during traversal according to the documented consistency. Aggregate methods such as `size`, `isEmpty`, and bulk reductions can be transient under concurrent mutation; use them for monitoring or estimates, not authorization decisions requiring a global snapshot.

> **HotSpot note:** Mainstream JDK implementations use CAS, bins, selective locking, resizing cooperation, and tree-shaped bins under collision for `ConcurrentHashMap`. These structures and thresholds are implementation details, not the Map or concurrency specification.

`CopyOnWriteArrayList` stores an array snapshot. Reads and traversal avoid mutation interference; an iterator sees the array version from iterator creation and does not support mutating iterator operations. Every

structural write copies the backing array, making writes O(n) and producing garbage. It fits small, read-mostly listener/config lists with rare changes, not event queues or frequently updated registries.

`ConcurrentLinkedQueue` is an unbounded nonblocking FIFO queue. `offer` and `poll` are efficient under concurrency, while `size()` typically traverses and can be expensive/inexact as a control signal under mutation. Because it is unbounded, it supplies no backpressure.

Blocking queues combine storage with wait/signal protocols:

- `ArrayBlockingQueue`: fixed capacity array, optional fairness, predictable bound.
- `LinkedBlockingQueue`: optionally bounded; default effectively large capacity is an overload risk.
- `SynchronousQueue`: no element capacity; each handoff pairs producer and consumer.
- `PriorityBlockingQueue`: priority ordering but logically unbounded capacity.
- `DelayQueue`: elements become available after delay expiration and is logically unbounded.

Methods express overload policy: `add` throws on full, `offer` reports immediately, timed `offer` waits to a deadline, and `put` waits interruptibly. Consumers similarly choose `poll`, timed `poll`, or `take`. Queue memory-consistency effects publish actions before insertion to a thread that subsequently removes/accesses the element through the specified handoff.

`ConcurrentSkipListMap` and Set provide sorted concurrent navigation with expected O(log n) search/update. They are useful when range queries and order are requirements; a hash-based structure is usually simpler/faster for unordered exact lookup. `ConcurrentSkipListMap` also disallows null.

Virtual threads are ordinary Java `Thread` instances in API semantics: they have names/IDs, interruption, thread locals, stack traces, and happens-before lifecycle rules. Java 21 creation includes:

JAVA

```
Thread.startVirtualThread(task);
Thread.ofVirtual().name("fetch-", 0).start(task);
Executors.newVirtualThreadPerTaskExecutor();
```

The executor creates a new virtual thread for each task; it is not a fixed pool of reusable virtual threads. Pooling cheap task threads reintroduces queueing and thread-local leakage without protecting a scarce resource. Bound the scarce resource directly with a connection pool, semaphore, bounded queue, rate limiter, or downstream concurrency control.

Virtual threads improve scalability for workloads with many blocking waits and a natural thread-per-request style. They do not increase CPU parallelism. CPU-bound work still competes for cores and may need an executor sized for CPU scheduling or a semaphore limiting expensive sections.

In Java 21, a virtual thread can be pinned to its carrier while executing inside a `synchronized` block/method or native/foreign call. If it blocks while pinned, the carrier cannot run another virtual thread. Brief uncontended synchronized sections are not inherently a problem; long/blocking operations while holding a monitor are. `ReentrantLock` waits integrate with parking and can be an alternative after measurement, but mechanical replacement can introduce bugs.

JDK 24 delivered JEP 491, which removes nearly all pinning caused by `synchronized` monitors. Selected native/foreign-call and class-loading or class-initialization situations can still retain a carrier. Code, diagnosis, and interview answers must therefore name the deployed release. Pinning affects scalability, not monitor correctness.

Thread locals work with virtual threads but can multiply memory by enormous thread counts. Large per-thread caches and pool-era assumptions should be removed. Scoped values and structured concurrency appear as preview APIs in Java 21 and must be compiled/run with preview enabled; they should not be presented as final Java 21 contracts.

> **Specification boundary:** Java 21 specifies virtual-thread API behavior and concurrent collection contracts. Carrier scheduling, continuation representation, default scheduler parallelism, ConcurrentHashMap layout, and specific pinning diagnostics are implementation details.

TRACE IT | Worked Java example

JAVA (continued 1/2)

```java
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.Semaphore;

public final class VirtualThreadQueries {
    private final Semaphore databaseSlots = new Semaphore(20);

    String query(int id) throws InterruptedException {
        databaseSlots.acquire();
        try {
            Thread.sleep(25); // Simulated interruptible blocking database call.
            return "row-" + id;
        } finally {
            databaseSlots.release();
        }
```

JAVA (continued 2/2)

```
    }

    List<String> loadAll(int count) throws InterruptedException, ExecutionException {
        try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<String>> futures = new ArrayList<>();
            for (int i = 0; i < count; i++) {
                int id = i;
                futures.add(executor.submit(() -> query(id)));
            }

            List<String> results = new ArrayList<>(count);
            for (Future<String> future : futures) {
                results.add(future.get());
            }
            return results;
        }
    }
}
```

The executor can represent hundreds or thousands of waiting tasks without an equally large platform-thread pool. The semaphore preserves the real database concurrency budget at 20. In production, the connection pool may itself enforce a bound, but an earlier application bulkhead can bound waiters and deadline handling more intentionally.

TRACE IT | Execution or memory walkthrough

For `loadAll(100)`:

1. The loop creates 100 task submissions; the executor starts one virtual thread per task rather than queuing behind a fixed virtual-thread pool.
2. The first 20 acquire semaphore permits. The remaining 80 park waiting for permits. Parking through the concurrency framework normally allows their carriers to run other virtual threads.
3. Each admitted thread calls the simulated blocking operation. `Thread.sleep` unmounts a non-pinned virtual thread in the Java 21 runtime, freeing its carrier.
4. When one sleep completes, the virtual thread is made runnable, remounts on an available carrier, returns its row, and releases a permit in `finally`.
5. A waiting virtual thread acquires that permit and proceeds. At no point does the simulated database have more than 20 operations.
6. `Future.get` consumes results in submission order. Later futures can already be complete, but one slow early future delays result assembly. A completion-order design could reduce head-of-line waiting if ordering is unnecessary.
7. If `get` is interrupted, try-with-resources closes the owned executor during unwinding under its API behavior, but production code still needs clear cancellation/deadline policy for outstanding tasks and external calls.

Each virtual thread has stack/task state, so 100 is not free. It is substantially lighter than reserving 100 large platform stacks, but captured request objects, thread locals, futures, and queue nodes still consume heap.

Complexity and performance

Submitting and collecting `n` tasks uses O(n) futures/result references in the example. Each semaphore operation is O(1) algorithmically but can wait. Overall elapsed time is approximately `ceil(n/20) * serviceTime` under the simplified equal-duration model, plus scheduling overhead.

Collection choices:

Structure	Typical read	Typical update	Iteration model	Capacity
`ConcurrentHashMap`	expected O(1)	expected O(1)	weakly consistent	unbounded by API
`ConcurrentSkipListMap`	expected O(log n)	expected O(log n)	weakly consistent, ordered	unbounded
`CopyOnWriteArrayList`	O(1) index	O(n) copy	snapshot	unbounded
`ConcurrentLinkedQueue`	O(1) offer/poll	O(1) expected	weakly consistent	unbounded
`ArrayBlockingQueue`	O(1)	O(1)	weakly consistent	fixed

Virtual-thread throughput benefits appear when tasks spend substantial time blocking. For pure computation, more runnable virtual threads increase scheduling overhead without adding cores. Pinning or synchronized native libraries can reduce carrier availability; JFR and load tests should verify.

WATCH OUT | Edge cases and common mistakes

- Composing individually thread-safe map calls into a racy compound operation.
- Expecting a concurrent iterator to produce an atomic whole-map snapshot.
- Calling `size()` repeatedly on `ConcurrentLinkedQueue` for admission.
- Using `CopyOnWriteArrayList` for frequent writes or large lists.
- Treating default `LinkedBlockingQueue` or priority/delay queues as safely bounded.
- Performing slow recursive work inside `ConcurrentHashMap.compute`.
- Pooling virtual threads instead of limiting the real resource.
- Assuming virtual threads make database connections or CPU unlimited.
- Holding a Java 21 monitor across slow blocking I/O and ignoring pinning.
- Carrying large thread-local caches into millions of virtual threads.
- Assuming virtual threads remove the need for interruption, deadlines, or shutdown.
- Using preview structured-concurrency/scoped-value APIs without labeling and enabling preview.

Production engineering notes

Choose a collection with an explicit consistency statement. If a dashboard may tolerate a weakly consistent traversal, document it. If billing requires a snapshot, build an immutable version, take a lock, use a database snapshot, or design an event log; do not infer snapshot semantics from "concurrent."

Incident example: a service migrates from a 100-thread JDBC executor to virtual-thread-per-request. Throughput rises briefly, then database latency and connection wait explode because 8,000 requests concurrently reach the 100-connection pool. Virtual threads made waiting cheap but did not make the database faster. Add admission aligned with connection/query capacity, propagate request deadlines, shed overload, and measure semaphore/connection wait separately from query time.

For Java 21 pinning, JFR can record virtual-thread pinning events, and HotSpot offers version-specific diagnostics such as pinned-thread tracing properties. Capture stack context and duration; prioritize blocking while pinned, not every short monitor. JDK 24's JEP 491 removes nearly all monitor pinning and changes the diagnostic picture, so confirm the exact JDK before applying a Java 21 runbook.

Monitor virtual-thread count, runnable/parked distribution, carrier utilization, task latency, downstream permit/connection waits, rejections, memory per task, and thread-local growth. A massive thread dump is hard to read; JFR and aggregate tooling are designed to group virtual-thread behavior.

TRY IT | Interview questions and model answers

Is `ConcurrentHashMap` enough for `containsKey` then `put`?

No. Each call is safe but the pair is not atomic. Use `putIfAbsent`, `computeIfAbsent`, `compute`, or another operation matching the transition. A multi-key invariant may still require locking or a different state model.

What does weakly consistent iteration mean?

Iteration tolerates concurrent modification and does not fail fast. It traverses elements according to the collection's documented consistency and may reflect some concurrent updates, but it is not an atomic snapshot.

Why are virtual threads useful?

They make thread-per-task blocking designs scalable by allowing blocked virtual threads to unmount from carrier platform threads. They improve concurrency for waiting-heavy workloads, not CPU parallelism or downstream capacity.

Should virtual threads be pooled?

Usually no. They are intended to be cheap per-task threads. Pooling them creates an artificial queue. Bound scarce resources such as connections, CPU-heavy sections, or partner calls directly.

What is pinning in Java 21?

A virtual thread can remain attached to a carrier while inside a monitor or native/foreign call. If it blocks then, the carrier cannot run another virtual thread, reducing scalability. Correctness remains, and later JDK behavior must be checked separately.

TRY IT | Exercises

1. Replace a `get/put` cache race with `computeIfAbsent`; specify loader failure behavior.
2. Compare weakly consistent and copy-on-write snapshot iteration in a listener registry.
3. Choose a blocking queue and overload method for a lossless worker, best-effort telemetry, and direct handoff.
4. Add a deadline-aware semaphore acquisition to the worked example.
5. Produce completion-order results instead of submission-order waits.
6. Create a Java 21 pinning demonstration safely, record it with JFR, then shorten the monitor scope.

RECAP | Chapter summary

Concurrent collections make documented individual and compound operations safe; they do not grant transactions across arbitrary call sequences. Iteration consistency and capacity are first-class selection criteria. Java 21 virtual threads let blocking tasks use a thread-per-task style by multiplexing execution over carriers, but CPU and downstream resources remain finite. Do not pool virtual threads; bound the scarce resource, monitor task memory, and diagnose Java 21 pinning where blocking occurs under monitors or native calls.

TRY IT | Revision checklist

- [ ] I can replace racy map compositions with an atomic operation.
- [ ] I distinguish weakly consistent and snapshot iteration.
- [ ] I know capacity/backpressure properties of major concurrent queues.
- [ ] I can explain virtual threads, carriers, mounting, and blocking.
- [ ] I do not equate concurrency with parallelism or downstream capacity.
- [ ] I understand Java 21 pinning and version dependence.
- [ ] I can design a virtual-thread service with explicit bulkheads and deadlines.

SDE-2 CORE CHAPTER

Chapter 7 - Concurrency Failure Modes, Testing, and Design Patterns

Learning objectives

- Classify races, deadlocks, livelocks, starvation, missed signals, leaks, and performance collapse.
- Diagnose blocking and progress failures from timelines, thread dumps, JFR, and capacity metrics.
- Test concurrent code with coordinated starts, invariants, histories, stress harnesses, and bounded completion.
- Apply immutability, confinement, message passing, lock ordering, open calls, and bulkheads.
- Separate safety, liveness, and performance claims in design and interviews.

WHY IT WORKS | Why this matters at SDE-2

Concurrent failures are nondeterministic only from the observer's perspective. They follow a protocol and a schedule. Adding sleeps may hide the schedule; adding logging may alter it. A test that passed one million times increases confidence but does not prove correctness. A thread dump showing every worker waiting may represent healthy idleness, a deadlock, downstream saturation, or same-pool starvation.

SDE-2 engineers turn symptoms into a graph: who owns which resource, who waits for whom, what state transition was expected, which happens-before edge is missing, and what progress guarantee was intended. They also prefer designs whose proof is short.

WHY IT WORKS | First-principles model

Evaluate concurrency across three independent properties:

- **Safety:** Nothing bad happens. Examples: no negative balance, no duplicate lease owner, linearizable queue operations.
- **Liveness:** Something good eventually happens. Examples: every accepted task completes or cancels; locks are eventually acquired under stated assumptions.
- **Performance:** It happens within resource and latency goals.

A design can be safe but deadlocked, live but too slow, or fast in a benchmark but racy.

Wait-for graphs make deadlock concrete:

```text
Thread T1 owns Lock A and waits for Lock B
Thread T2 owns Lock B and waits for Lock A

T1 -> B -> T2 -> A -> T1     cycle means no participant can proceed
```

Removing one necessary condition prevents classic lock deadlock: mutual exclusion, hold-and-wait, no preemption, or circular wait. Lock ordering targets circular wait.

Core terminology

- **Race condition:** Result depends incorrectly on relative timing/interleaving.
- **Data race:** Conflicting accesses not ordered by happens-before.
- **Lost update:** Two read-modify-write operations overwrite one effect.
- **Deadlock:** Participants wait in a cycle or condition that cannot be satisfied.
- **Livelock:** Participants keep changing/retrying but make no useful progress.
- **Starvation:** One participant is indefinitely denied progress while others proceed.
- **Priority inversion:** Higher-priority work waits behind lower-priority work holding a needed resource.
- **Thread starvation deadlock:** All workers wait for tasks queued to the same exhausted executor.
- **Linearizability:** Concurrent history behaves as if each operation took effect atomically between call and response.
- **Stress test:** Repeated/concurrent exploration intended to expose allowed schedules.

Detailed mechanics

Data races create visibility and ordering uncertainty. Race conditions are broader: two correctly atomic operations can still violate a check-then-act protocol. Lost updates, duplicate initialization, stale snapshots, and unsafe publication belong here. The fix is to make the intended transition atomic and ordered, not merely mark every field volatile.

Missed-signal bugs occur when condition state and notification are not protected by one protocol. If a producer signals before a consumer enters an unrelated wait, the consumer can sleep forever despite available work. A correct condition variable changes/checks the predicate under one lock and always waits in a loop. Semaphores, latches, futures, and blocking queues retain state and are often safer than hand-written notifications.

Classic lock deadlock often meets four conditions:

1. Resources have exclusive ownership.
2. A participant holds one while waiting for another.
3. Ownership cannot be forcibly taken safely.

4 The wait-for graph contains a cycle.

Prevent it with a global lock order, one coarser lock, atomic database operation, avoiding nested ownership, or timed/interruptible acquisition plus rollback. Timeouts detect/escape some waits but do not make a protocol correct: repeated partial work can create livelock or data inconsistency.

Livelock example: two polite transfer workers detect conflict, both release, sleep for the same fixed delay, and retry in sync forever. Randomized/exponential backoff can reduce synchronization, but a deterministic owner/order is preferable when possible. Starvation can arise from unfair locks, endless high-priority tasks, read-heavy read/write locks, CAS losers, or a scheduler/resource admission policy.

Thread starvation deadlock does not require two locks. In a two-thread executor, two parent tasks submit child tasks to the same executor and block on their futures. Both workers are occupied by parents; children remain queued. Compose without blocking, run child work directly, allocate a separately justified executor, or redesign ownership.

Cancellation failures are liveness/resource bugs. A timeout on a caller does not necessarily stop downstream work. Orphan tasks can later mutate state, retain memory, hold connections, or consume the result of an expired request. Define cancellation propagation, idempotency, deadlines, and what commits are still legal after caller departure.

Thread leakage occurs when threads/executors are created repeatedly and never terminated, or when blocked tasks never receive cancellation. Thread-local leakage occurs when pooled threads retain per-request data after completion. Always remove manually managed thread locals in `finally`; prefer explicit context passing.

Testing must control setup without pretending to control the runtime schedule. Use `CountDownLatch`, `CyclicBarrier`, or `Phaser` to align starts and increase overlap. Use timeouts so a deadlock fails the test rather than hangs the suite. Avoid sleeps as correctness synchronization; a sleep can be a workload delay but should not prove that another thread finished.

Assert invariants and histories, not just final happy values. For a transfer system, total balance and nonnegative balances matter after every legal transition. For a concurrent object, record invocation/response times and results, then check whether a legal sequential ordering exists between them. This is linearizability testing; exhaustive checking can be expensive, so histories are bounded.

The OpenJDK jcstress project embodies JVM concurrency stress-testing concepts: actors perform operations concurrently, an arbiter observes outcomes, and results are classified as acceptable, interesting, or forbidden. It is especially useful for JMM litmus tests. Repeating a JUnit method manually is less rigorous because compiler warm-up, actor placement, outcome collection, and fork isolation matter. Stress tools find counterexamples; they do not prove all large programs correct.

Model checking or deterministic scheduler frameworks can explore controlled interleavings for abstractions they support. Static analysis can find inconsistent locking and unsafe publication patterns. Code review remains essential: list shared mutable variables, all accesses, guards, linearization points, cancellation points, and progress assumptions.

Design patterns reduce states:

- **Immutability plus atomic publication:** replace one validated snapshot.
- **Thread confinement/single writer:** one owner mutates state; others send messages.

- **Producer-consumer:** bounded queue separates producers and workers with backpressure.
- **Bulkhead:** independent concurrency limit per dependency/workload.
- **Lock ordering:** acquire multiple locks by a stable global key.
- **Open call:** copy/decide under lock, invoke unknown or slow code after releasing it.
- **Split phase:** begin asynchronous work, release resources, resume on completion.
- **Idempotent command:** retries/cancellation races do not duplicate business effects.
- **Structured lifetime:** child work does not silently outlive its owning operation.

Structured concurrency is a preview API in Java 21, not a finalized Java 21 feature. Its design idea remains useful: task lifetimes form a tree, sibling failure/cancellation is coordinated, and the owner joins before leaving scope.

> **Specification boundary:** Java defines monitor, volatile, atomic, thread lifecycle, and concurrent-library contracts. It does not guarantee fair scheduling or that stress tests explore every legal execution. Deadlock freedom and linearizability are properties the program must establish.

TRACE IT | Worked Java example

JAVA (continued 1/2)

```java
import java.util.concurrent.locks.ReentrantLock;

public final class BankTransfers {
    static final class Account {
        final long id;
        final ReentrantLock lock = new ReentrantLock();
        long cents;

        Account(long id, long cents) {
            if (cents < 0) throw new IllegalArgumentException("cents");
            this.id = id;
            this.cents = cents;
        }

        long balance() {
            lock.lock();
            try {
                return cents;
            } finally {
                lock.unlock();
            }
        }
    }

    static boolean transfer(Account from, Account to, long cents)
```

JAVA (continued 2/2)

```
            throws InterruptedException {
        if (from == to) return true;
        if (cents <= 0) throw new IllegalArgumentException("cents");
        if (from.id == to.id) throw new IllegalArgumentException("duplicate id");

        Account first = from.id < to.id ? from : to;
        Account second = from.id < to.id ? to : from;

        first.lock.lockInterruptibly();
        try {
            second.lock.lockInterruptibly();
            try {
                if (from.cents < cents) return false;
                from.cents -= cents;
                to.cents += cents;
                return true;
            } finally {
                second.lock.unlock();
            }
        } finally {
            first.lock.unlock();
        }
    }
}
```

Every transfer acquires accounts by ascending stable ID regardless of direction, eliminating a two-account circular wait. Both balance changes occur while both locks are held, so the in-process invariant moves atomically relative to code following the same discipline.

TRACE IT | Execution or memory walkthrough

Let account A have ID 1 and B have ID 2. T1 transfers A to B while T2 transfers B to A:

1. Both compute `first=A`, `second=B` because ordering depends on IDs, not direction.
2. Suppose T1 acquires A. T2 waits interruptibly for A and owns no other account lock.
3. T1 acquires B, validates funds, subtracts/adds, releases B then A.
4. Lock release/acquisition orders the updated balances for T2.
5. T2 acquires A then B and performs its transition.

No state has T1 holding A waiting for B while T2 holds B waiting for A. The wait-for graph cannot contain that two-lock cycle if every operation respects the same total order.

If T1 is interrupted while waiting for B, `lockInterruptibly` throws. T1 never acquired B, so the inner `try/finally` did not begin. The outer finally releases A. No balances changed because mutation follows both acquisitions.

Overflow is not handled in this educational code. `to.cents += cents` should use exact arithmetic or validated limits in a financial implementation. Across multiple JVMs/database rows, these Java locks do not coordinate other processes; a database transaction or distributed protocol is required.

Complexity and performance

Transfer does $O(1)$ domain work but lock wait is unbounded without stronger scheduling assumptions. Contention is per account: unrelated account pairs can proceed concurrently. A global bank lock simplifies proof but serializes all transfers; fine-grained account locks increase parallelism and lock-order complexity.

Deadlock detection from a wait-for graph is $O(V + E)$ for cycle detection, but production evidence can omit application-level resources such as database locks or message acknowledgments. Tail latency increases nonlinearly near resource saturation even without deadlock.

Stress testing cost grows with threads, operations, histories, and possible interleavings. Complete schedule exploration is generally exponential. Target small state machines, boundary races, and known weak points, then combine formal reasoning, static checks, unit tests, stress, integration load, and production observability.

WATCH OUT | Edge cases and common mistakes

- Concluding "no race" because a test passed repeatedly.
- Fixing a race by adding sleep, logging, or `yield`.
- Taking one thread dump and declaring deadlock without an ownership cycle.
- Applying lock ordering in one method while another path violates it.
- Generating lock-order IDs that can collide or change.
- Timing out lock acquisition but leaving partial side effects before retry.
- Retrying conflicts with identical delay, creating livelock.
- Using fair locks as a complete starvation guarantee.
- Blocking executor workers on child tasks queued to the same executor.
- Letting timed-out work commit non-idempotent effects later.
- Forgetting to shut down executors or clear thread locals.
- Treating a synchronized in-memory invariant as distributed consistency.
- Testing only final count and missing illegal intermediate states.

Priority changes are not a portable correctness fix for priority inversion. OS/JVM mappings vary, and application priorities can worsen starvation. Reduce lock duration, separate workloads, and use explicit admission.

Production engineering notes

Diagnose from a synchronized timeline:

1. Determine whether the symptom is wrong state, no progress, or slow progress.
2. Capture several thread dumps, not just one; include timestamps and CPU/load.
3. Identify RUNNABLE CPU consumers, monitor owners/contenders, parked explicit-lock waiters, executor queues, and downstream waits.
4. Use JVM deadlock detection where available, but add database lock graphs, connection pools, queue lag, and remote dependency state.
5. Correlate JFR lock/park events, CPU profiles, GC/safepoints, and request traces.
6. Preserve the triggering workload and deploy a protocol fix with a regression stress test.

Incident example: all 32 pool threads show `FutureTask.get`, CPU is near zero, and queue depth grows. Each task submitted an enrichment subtask to the same pool and waited. No Java monitor cycle appears, so automatic deadlock detection reports nothing. This is thread-starvation deadlock. Replace blocking nested submission with completion composition or reserve/avoid the dependency structure; adding workers only moves the threshold.

Livelock evidence looks different: CPU and retry metrics are high, state changes repeatedly, but completed operations remain near zero. Add correlation IDs and attempt counts with rate-limited logs, then establish ownership/order or randomized bounded backoff.

> **HotSpot note:** HotSpot thread dumps and `jcmd Thread.print` can identify owned/waited monitors and some ownable synchronizers; JFR can capture Java monitor enter/wait, park, virtual-thread, and scheduling-related evidence depending on JDK configuration. Tool visibility is not a proof that no application or external deadlock exists.

TRY IT | Interview questions and model answers

Deadlock versus livelock versus starvation?

In deadlock, participants are blocked in a cycle or unsatisfiable wait. In livelock, they keep reacting/retrying but complete no useful work. In starvation, the system progresses while one participant is indefinitely denied resources. Diagnosis and remedies differ.

How do you prevent deadlock?

Remove a necessary condition: avoid nested locks, use one lock, acquire all locks in a stable global order, use timed/interruptible acquisition with safe rollback, or redesign around message passing/transactions. Then verify every acquisition path follows the protocol.

How do you test concurrent code?

First prove invariants and happens-before/linearization points. Use latches or barriers to align actors, bounded timeouts to catch hangs, record histories, and stress across many forks/architectures with a harness such as jcstress for JMM cases. Tests find counterexamples but do not replace proof.

What is thread-starvation deadlock?

All executor workers block waiting for tasks that are queued to the same executor, leaving no worker to run them. It may not appear as a monitor cycle. Avoid blocking same-pool dependency chains and compose work or redesign capacity.

Which concurrency design patterns do you prefer?

Start with immutability, confinement/single ownership, immutable snapshot publication, bounded message passing, and high-level concurrent operations. Use locks for explicit multi-field invariants, with stable ordering and open calls. The best design minimizes shared mutable states.

TRY IT | Exercises

1. Write a two-lock deadlock, capture it safely, then impose a global order.
2. Build a same-executor parent/child starvation example and replace blocking gets with composition.
3. Design a barrier-based lost-update test with acceptable and forbidden outcomes.
4. Define a linearizable history checker for a tiny concurrent stack with at most four operations.
5. Refactor a callback-under-lock component using the open-call pattern.
6. Design cancellation propagation for an HTTP request, database query, and message publish with an idempotency key.
7. List safety, liveness, and performance requirements for a bounded work queue separately.

RECAP | Chapter summary

Concurrency correctness requires separate safety, liveness, and performance arguments. Races violate atomicity or ordering; deadlocks form unsatisfiable waits; livelocks spend work without progress; starvation denies one participant. Tests should coordinate actors, assert invariants/histories, use deadlines, and stress with purpose, while recognizing that absence of a failure is not proof. Immutability, confinement, bounded message passing, lock ordering, open calls, bulkheads, and structured lifetimes reduce the state space and operational risk.

TRY IT | Revision checklist

- [ ] I can distinguish data races, race conditions, lost updates, and missed signals.
- [ ] I can draw a wait-for graph and apply a stable lock order.
- [ ] I can identify deadlock, livelock, starvation, and same-pool starvation evidence.
- [ ] I use barriers/latches and bounded timeouts rather than sleeps for tests.
- [ ] I understand linearizability histories and jcstress-style outcome testing.
- [ ] I can design cancellation and shutdown without orphan work.
- [ ] I prefer immutability, confinement, message passing, and explicit ownership.
- [ ] I separate in-process synchronization from distributed consistency.

SDE-2 CORE CHAPTER

Chapter 8 - Concurrency Invariants, Cancellation, and Capacity Lab

Concurrency code is not correct because it uses a class from `java.util.concurrent`. It is correct when every shared invariant has an ownership rule, every cross-thread observation has a happens-before path, every task has a completion/cancellation policy, and admitted work fits a resource budget.

Use this review order:

TEXT

```
state -> invariant -> owners -> transition -> happens-before edge
      -> progress/cancellation -> admission bound -> evidence
```

If you start by asking "volatile or synchronized?", you are choosing a mechanism before defining the problem.

Step 1: classify the shared state

Thread-confined

One thread owns the mutable value and does not publish it. No synchronization is needed for that state. Confinement can be lexical, request-scoped, actor/event-loop ownership, or task ownership.

Immutable and safely published

Construct a complete value, validate it, and publish one reference through a happens-before edge. Readers take one snapshot and use it for the whole operation.

JAVA

```java
record Config(String endpoint, int timeoutMillis) {}

private volatile Config current;

void replace(Config next) {
    current = validate(next);
}
```

Two separate volatile fields for endpoint and timeout would allow mixed versions when a refresh happens between reads.

Shared mutable with one invariant

Examples include balance plus version, queue plus capacity, or lifecycle state plus owner. Protect the complete transition with one lock or represent it as one immutable state behind one atomic reference.

Independent statistics

Metrics that tolerate a non-atomic aggregate can use striped accumulators such as `LongAdder`. Do not transfer that choice to an exact balance, sequence, or inventory count.

WHY IT WORKS | Step 2: write the invariant as a sentence

Examples:

- `available >= 0` and version increases exactly once per successful reservation;
- queued plus running work never exceeds the configured admission boundary;
- once shutdown starts, no new externally owned task is accepted;
- a published snapshot's endpoint and timeout come from the same version;
- locks are acquired only in global order `(accountId ascending)`.

An invariant should be checkable at the transition point. "Thread-safe" is a conclusion, not an invariant.

Step 3: prove visibility with happens-before

Happens-before is not wall-clock order. It is the model's guarantee that one action's effects are visible and ordered before another action.

Common edges:

- actions before `Thread.start()` happen-before actions in the started thread;
- actions in a thread happen-before another thread successfully returns from `join()`;
- an unlock happens-before a later lock of the same monitor/lock;
- a write to a volatile variable happens-before a subsequent read of that variable in synchronization order;
- library synchronizers such as latches, futures, queues, and executors define memory-consistency effects in their contracts.

Publication proof

TEXT

```
writer program order:
answer = 42
ready = true   (volatile write)
       |
       | happens-before
       v
reader reads ready == true (volatile read)
observed = answer
```

The ordinary `answer` write is published through the volatile edge and transitivity. The volatile flag does not make `answer++` atomic, and it does not protect a larger invariant if later writes mutate the published object unsafely.

Step 4: choose a transition mechanism

synchronized or a lock

Use a lock when a transition spans multiple fields/structures, must wait on a condition, or benefits from one obvious critical section.

JAVA

```java
synchronized void transfer(Account target, long cents) {
    // One monitor is insufficient if target has independent ownership.
}
```

For two independently locked accounts, define a global acquisition order. Locking source then target can deadlock when another thread transfers in the opposite direction.

Intrinsic locks release automatically on exceptional exit. `ReentrantLock` requires `unlock()` in `finally`. Its additional capabilities-interruptible acquisition, timed try, conditions, optional fairness-are useful only when the protocol needs them.

Atomic read-modify-write

`AtomicInteger.incrementAndGet()` is one atomic transition on one location. `volatile int count; count++` is still read, add, write and can lose updates.

A CAS loop:

TEXT

```
read observed state
derive side-effect-free proposal
CAS(observed, proposal)
  success -> linearization point
  failure -> reread and recompute
```

The proposal function may run repeatedly. Do not charge a card, send a message, or append to an external log inside it.

Atomics do not combine independent locations. If `used` and `limit` form one invariant, store one immutable state in one atomic reference or use a lock.

Concurrent collection compound operations

A concurrent collection makes its documented individual operations safe; a sequence of operations is not automatically atomic.

Weak:

```java
if (!map.containsKey(key)) {
    map.put(key, createValue());
}
```

Use the collection's compound operation when its semantics fit:

```java
map.computeIfAbsent(key, this::createValue);
```

The mapping function must obey that implementation's contract. Avoid slow external side effects or recursive updates, and do not assume it executes exactly once under every failure/retry scenario unless the API promises that.

Interruption is a request carried by state

`Thread.interrupt()` does not forcibly kill arbitrary Java code. It sets interrupt status and causes certain blocking methods to throw `InterruptedException`, often clearing status as they do so.

At a boundary, choose one policy:

1. propagate `InterruptedException` when the method contract allows it;
2. translate to a domain cancellation failure while restoring status;
3. if this layer owns the task, clean up and terminate promptly.

Do not swallow:

```java
try {
    queue.take();
} catch (InterruptedException ignored) {
    // broken: caller cancellation was lost
}
```

When translating:

JAVA

```java
} catch (InterruptedException interruption) {
    Thread.currentThread().interrupt();
    throw new TaskCancelledException("cancelled", interruption);
}
```

Restoring status is not ritual. Do it when higher layers still need to observe the request; do not restore and then continue a long loop that ignores it.

Cancellation must propagate through the whole call graph. Cancelling a `Future` cannot reliably stop an operation that ignores interruption or blocks in a non-interruptible external API.

Executor configuration is an admission policy

A pool has three separate questions:

- how many tasks may execute concurrently?
- how many may wait?
- what happens when both capacities are full?

TEXT

```
submit
  |
  +-> idle/core worker? -> run
  |
  +-> queue has room?   -> wait
  |
  +-> grow allowed?     -> new worker
  |
  `-> reject policy     -> explicit overload response
```

An unbounded queue can keep rejection out of logs while moving failure into latency and memory. A bounded queue makes overload visible. The bound should follow arrival bursts, service time, deadline, memory per task, and acceptable queue delay-not a copied number.

`CallerRunsPolicy` is not universal backpressure. It makes the submitting thread execute the task, which can slow producers, but it can also block an event loop, request thread, or lock holder. Evaluate the submission context.

Deterministic saturation example

With one worker and one queue slot:

1. task A occupies the worker and waits on a latch;
2. task B occupies the queue slot;
3. task C reaches the rejection policy;
4. release A, drain B, shut down, await termination.

This is a reliable test. Sleeping for "long enough" and hoping the pool is full is not.

Virtual threads increase concurrency, not downstream capacity

Virtual threads make thread-per-task style practical for many blocking workloads. They do not make CPU work faster, increase a database connection pool, or remove an external rate limit.

TEXT

```
100,000 virtual-thread tasks
        |
        v
database permits = 50
```

The application still needs a semaphore, bounded connection pool, rate limiter, queue, or other admission policy around the scarce dependency.

In the Java 21 baseline, some blocking while holding an intrinsic monitor and some native/foreign calls can pin a virtual thread's carrier. Later JDKs changed important pinning behavior, including monitor-related improvements. State the target JDK and confirm with JFR/runtime evidence instead of repeating one version's rule forever.

Thread-local values work with virtual threads but can multiply retained state when task counts are huge. Prefer explicit context where ownership and cleanup matter.

CompletableFuture: separate execution, completion, and failure

For every stage, ask:

- which executor runs it?
- does it transform a value (`thenApply`) or compose another stage (`thenCompose`)?
- how does failure propagate?
- who owns timeout/cancellation?
- can blocking inside the stage starve the chosen executor?

`exceptionally` recovers from failure with a value. `handle` observes both success and failure. `whenComplete` is usually for observation/side effects and does not inherently turn failure into success.

A timeout on a wrapper stage does not necessarily abort the underlying remote operation. Production cancellation needs a protocol at the I/O/client boundary.

Failure-mode matrix

Failure	Required conditions	Evidence	Design response
lost update	non-atomic read-modify-write	invariant counters, stress test	lock/atomic compound operation
stale read	data race/no publication edge	code-level HB analysis; controlled test	safe publication/synchronization
deadlock	cyclic wait-for graph	multiple thread dumps/lock owners	ordering, try-lock protocol, reduce nested ownership
starvation	one task repeatedly denied progress	per-task latency/queue/lock evidence	fairness/partitioning/shorter critical section
livelock	threads act but state never advances	CPU profile plus repeated state transitions	randomized/backoff/coordination redesign
executor overload	arrival exceeds service/admission	active, queue, rejection, deadline metrics	bound and shed/degrade/scale
cancellation leak	task ignores or cannot propagate stop	task age, outstanding futures, thread traces	explicit cancellation protocol
virtual-thread pinning	target-version pinning condition	JFR pinned-thread evidence	shorten monitor/native block or upgrade/redesign

Testing concurrency without pretending to prove absence of races

One passing run does not prove thread safety. Use layers:

1. **Deterministic protocol tests:** latches/barriers place operations at important states.
2. **Invariant stress tests:** many randomized schedules check postconditions repeatedly.
3. **Specialized tools:** jcstress-style tests for memory-model outcomes; static analysis where useful.
4. **Production evidence:** queue depth, rejection, task age, lock/park events, CPU, cancellation, and deadline metrics.

Avoid `Thread.sleep` as the synchronization mechanism in a correctness test. A sleep can make a race less likely on the fastest machine and more likely under CI load without proving the desired order.

WHY IT WORKS | Executable invariant companion

`ConcurrencyInvariantChecks.java` runs six deterministic suites:

- synchronized and atomic exact counters under concurrent start;
- volatile publication of an ordinary write;
- interruption of a blocking wait with restored status;
- bounded executor saturation and explicit rejection;
- Java 21 virtual-thread execution.

BASH

```
out=$(mktemp -d)
javac --release 21 -Xlint:all -Werror -d "$out" \
  content/volumes/java/JAVA-07-concurrency-and-memory-model/code/ConcurrencyInvariantChecks.java
java -ea -cp "$out" ConcurrencyInvariantChecks
```

Expected output:

TEXT

```
PASS 6 concurrency invariant suites
```

Timeouts in the harness are failure bounds, not ordering mechanisms. Latches establish the intended states.

Interview room: worked answers

Does volatile make an object thread-safe?

Model answer: A volatile reference safely orders replacement/publication of that reference. It does not make later mutations to the referenced object atomic or safe. I either publish an immutable snapshot and never mutate it, or protect the object's mutable invariant separately.

When do you prefer a lock over CAS?

Model answer: When the invariant spans multiple locations, the transformation is expensive, waiting/conditions are needed, or contention makes retries wasteful. CAS is attractive for a small one-location transition with a side-effect-free proposal. I compare correctness proof and tail latency, not "lock-free sounds faster."

What is the linearization point of a CAS reservation?

Model answer: The successful compare-and-set that replaces observed immutable state with proposed state. Failed proposals have no external effect and must be recomputed from a new observation.

How do you prevent transfer deadlock?

Model answer: Acquire account locks in one global order independent of transfer direction, release in reverse in `finally`, and define the equal-key case. Alternatively centralize ownership or perform a transactional conditional update. Timeouts detect or bound waiting but do not alone prove correctness.

How do you size a thread pool?

Model answer: I begin with workload type, measured service time, arrival rate/bursts, downstream concurrency, task memory, and deadline. I bound queueing and define rejection/degradation. A formula can seed a test, but production metrics validate the policy.

Are virtual threads a replacement for a connection pool?

Model answer: No. They reduce the cost of blocked thread-per-task code. The database still has finite connections and throughput, so I keep an explicit downstream admission bound.

Should an InterruptedException always be rethrown?

Model answer: It should not be silently lost. A boundary can propagate it, translate after restoring status, or consume it when that layer owns termination and actually stops. The policy must preserve cancellation through the call graph.

Why can a timeout leave work running?

Model answer: A timeout can complete a waiting stage exceptionally without cancelling the underlying operation, or cancellation can rely on interruption that the operation ignores. I design timeout and cancellation at the resource/client boundary and observe outstanding work after callers leave.

TRY IT | Exercises

1. **Foundation:** Draw the happens-before chain for a configuration snapshot published through one volatile reference.
2. **Debugging:** Repair `volatile int count; count++` for an exact counter and state the linearization point.
3. **Interview Core:** Protect a balance/version invariant with both a lock and an atomic immutable state; compare proofs.
4. **Debugging:** Replace sleep-based executor saturation with latches and assert rejection deterministically.
5. **SDE-2 Follow-up:** Design cancellation through controller, service, executor task, and HTTP client.
6. **SDE-2 Follow-up:** Bound 20,000 virtual-thread requests against 50 database connections and define overload behavior.
7. **Production:** Diagnose intermittent request stalls from three thread dumps and queue/lock metrics.

Worked solutions

1. Writer constructs and validates the complete immutable config, performs a volatile reference write; reader performs a subsequent volatile read and uses that one local reference. Program order plus volatile synchronization plus transitivity publishes all constructor writes.
2. Use `AtomicInteger.incrementAndGet()` for one exact counter or a lock when it belongs to a larger invariant. The atomic operation or locked critical-section transition is the linearization point; volatile alone does not combine read and write.
3. A lock protects both fields in one critical section and can validate/transition in place. An atomic reference stores both in one immutable value and linearizes at successful CAS; the proposal must be side-effect free and can allocate/retry. Choose by contention and surrounding operations.
4. Occupy the only worker with task A blocked on a latch, wait until A signals running, enqueue B into the one-slot queue, submit C and assert the rejection policy, then release and terminate. No timing guess is needed.
5. Define one request cancellation token/deadline, cancel owned futures, propagate interruption, configure client request/connect/read deadlines, and ensure response timeout does not orphan remote work. Record cancellation reason and outstanding-task age.
6. Keep virtual thread per request if it simplifies code, but acquire a 50-permit dependency admission guard with bounded wait tied to the request deadline. Reject or degrade when permits cannot be obtained; expose wait and rejection metrics.
7. Build a wait-for picture from repeated dumps. If the same lock owner is blocked on a downstream call while many request threads queue behind it, shorten/move I/O outside the critical section. Correlate with queue age and downstream latency before changing pool size.

Final checklist

- I define ownership and invariants before choosing primitives.
- I can draw the exact happens-before path for publication.
- I distinguish visibility, atomicity, ordering, and progress.
- I preserve interruption/cancellation policy across boundaries.
- I treat executors and virtual threads as admission decisions.
- I use deterministic scheduling controls in tests.
- I diagnose from waits, queues, owners, and timelines rather than thread count alone.

SDE-2 CORE CHAPTER

Chapter 9 - Concurrency Invariant Checks

Deterministic Java 21 checks for synchronized and atomic transitions, volatile publication, interruption, bounded executor rejection, and virtual-thread execution. A successful run prints exactly PASS 6 concurrency invariant suites.

JAVA (continued 1/6)

```java
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.RejectedExecutionException;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;

/** Deterministic checks for concurrency invariants and lifecycle policy. */
public final class ConcurrencyInvariantChecks {
    private ConcurrencyInvariantChecks() {}

    public static void main(String[] args)
            throws InterruptedException, ExecutionException {
        verifySynchronizedTransition();
        verifyAtomicTransition();
        verifyVolatilePublication();
        verifyInterruptionProtocol();
        verifyBoundedExecutorAdmission();
        verifyVirtualThreadTask();
        System.out.println("PASS 6 concurrency invariant suites");
    }

    private static void verifySynchronizedTransition() throws InterruptedException {
        LockedCounter counter = new LockedCounter();
```

JAVA (continued 2/6)

```
        runConcurrently(4, 2_500, counter::increment);
        check(counter.value() == 10_000, "synchronized counter transition");
    }

    private static void verifyAtomicTransition() throws InterruptedException {
        AtomicInteger counter = new AtomicInteger();
        runConcurrently(4, 2_500, counter::incrementAndGet);
        check(counter.get() == 10_000, "atomic counter transition");
    }

    private static void runConcurrently(int threadCount, int iterations,
                                        Runnable operation) throws InterruptedException {
        CountDownLatch start = new CountDownLatch(1);
        List<Thread> workers = new ArrayList<>();
        for (int worker = 0; worker < threadCount; worker++) {
            Thread thread = Thread.ofPlatform().unstarted(() -> {
                awaitUninterruptiblyForTest(start);
                for (int iteration = 0; iteration < iterations; iteration++) {
                    operation.run();
                }
            });
            workers.add(thread);
            thread.start();
        }
        start.countDown();
        for (Thread worker : workers) {
            worker.join();
        }
    }

    private static void verifyVolatilePublication() throws InterruptedException {
```

JAVA (continued 3/6)

```
        Publication publication = new Publication();
        AtomicInteger observed = new AtomicInteger(-1);
        CountDownLatch readerStarted = new CountDownLatch(1);

        Thread reader = Thread.ofPlatform().start(() -> {
            readerStarted.countDown();
            while (!publication.ready) {
                Thread.onSpinWait();
            }
            observed.set(publication.answer);
        });

        check(readerStarted.await(5, TimeUnit.SECONDS), "reader started");
        publication.answer = 42;
        publication.ready = true;
        reader.join(5_000);

        check(!reader.isAlive() && observed.get() == 42,
                "volatile flag publishes prior ordinary write");
    }

    private static void verifyInterruptionProtocol() throws InterruptedException {
        CountDownLatch waiting = new CountDownLatch(1);
        CountDownLatch neverReleased = new CountDownLatch(1);
        AtomicBoolean restoredStatus = new AtomicBoolean();

        Thread worker = Thread.ofPlatform().start(() -> {
            try {
                waiting.countDown();
                neverReleased.await();
            } catch (InterruptedException interruption) {
```

JAVA (continued 4/6)

```
                Thread.currentThread().interrupt();
                restoredStatus.set(Thread.currentThread().isInterrupted());
            }
        });

        check(waiting.await(5, TimeUnit.SECONDS), "worker reached blocking call");
        worker.interrupt();
        worker.join(5_000);
        check(!worker.isAlive() && restoredStatus.get(),
                "interruption was handled and status restored");
    }

    private static void verifyBoundedExecutorAdmission() throws InterruptedException {
        CountDownLatch firstTaskRunning = new CountDownLatch(1);
        CountDownLatch releaseFirstTask = new CountDownLatch(1);
        ThreadPoolExecutor executor = new ThreadPoolExecutor(
                1,
                1,
                0L,
                TimeUnit.MILLISECONDS,
                new ArrayBlockingQueue<>(1),
                new ThreadPoolExecutor.AbortPolicy());

        try {
            executor.execute(() -> {
                firstTaskRunning.countDown();
                awaitUninterruptiblyForTest(releaseFirstTask);
            });
            check(firstTaskRunning.await(5, TimeUnit.SECONDS), "first task running");
            executor.execute(() -> { });
```

JAVA (continued 5/6)

```
            boolean rejected = false;
            try {
                executor.execute(() -> { });
            } catch (RejectedExecutionException expected) {
                rejected = true;
            }
            check(rejected, "third task rejected at explicit capacity boundary");
        } finally {
            releaseFirstTask.countDown();
            executor.shutdown();
            check(executor.awaitTermination(5, TimeUnit.SECONDS),
                    "bounded executor termination");
        }
    }

    private static void verifyVirtualThreadTask()
            throws InterruptedException, ExecutionException {
        try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
            Future<Boolean> virtual = executor.submit(
                    () -> Thread.currentThread().isVirtual());
            check(virtual.get(), "task ran on a virtual thread");
        }
    }

    private static void awaitUninterruptiblyForTest(CountDownLatch latch) {
        boolean interrupted = false;
        while (true) {
            try {
                latch.await();
                break;
            } catch (InterruptedException interruption) {
```

JAVA (continued 6/6)

```
                interrupted = true;
            }
        }
        if (interrupted) {
            Thread.currentThread().interrupt();
        }
    }

    private static void check(boolean condition, String message) {
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    private static final class LockedCounter {
        private int value;

        synchronized void increment() {
            value++;
        }

        synchronized int value() {
            return value;
        }
    }

    private static final class Publication {
        private int answer;
        private volatile boolean ready;
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: lifecycle and synchronization
- Interview Core: volatile, atomics, executors, and collections
- SDE-2 Follow-up: cancellation and backpressure
- Challenge: diagnose a race, deadlock, or starvation incident

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: JAVA 06 - JVM and Java Execution

Series index

Next book: JAVA 08 - Performance, Diagnostics, and GC Incidents

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Language, OOP, and Modern Java
Java	**JAVA 05**	Collections, Streams, and I/O
Java	**JAVA 06**	JVM and Java Execution
Java	**JAVA 07**	Java Concurrency and the Memory Model
Java	**JAVA 08**	Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Java Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
Frameworks	**FW 01**	MySQL for Java Backend Interviews
Frameworks	**FW 02**	Hibernate and JPA for Java Backend Interviews
Frameworks	**FW 03**	Spring Framework for Java Engineers
Frameworks	**FW 04**	Spring Boot for Java Backend Engineers
Frameworks	**FW 05**	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	**FW 06**	Spring Data for Java Backend Engineers
Frameworks	**FW 07**	MongoDB for Java Backend Interviews
Frameworks	**FW 08**	Redis for Java Backend Interviews
Frameworks	**FW 09**	Persistence, SQL, JPA, and Caching
Frameworks	**FW 10**	Apache Kafka and Spring Kafka
Frameworks	**FW 11**	Spring Ecosystem Extensions
Frameworks	**FW 12**	Spring AI for Java Engineers
System Design	**SD 01**	Design, Backend, Testing, and Security
System Design	**SD 02**	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Java Engineering - Book 07 of 09 - Study Step JAVA-07. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.